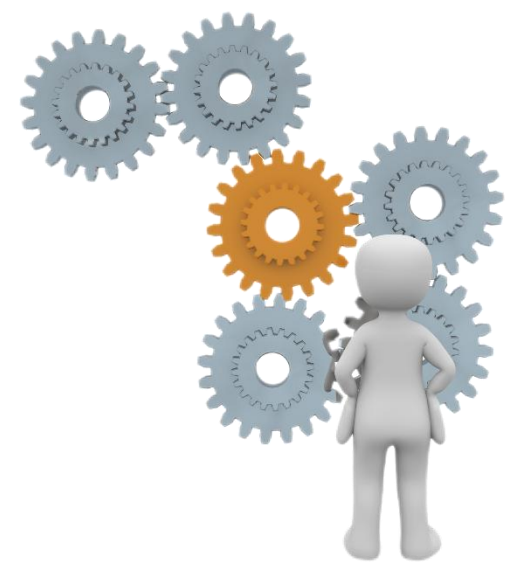


第5章 递归



5.1 什么是递归

5.1.1 递归的定义

在定义一个过程或函数时，出现直接或者间接调用自己的成分，称之为**递归**。

- 若直接调用自己，称之为**直接递归**。
- 若间接调用自己，称之为**间接递归**。
- 如果一个递归过程或递归函数中递归调用语句是最后一条执行语句，则称这种递归调用为**尾递归**。

5.1.1 递归的定义

示例：求 $n!$

解

- 求 $n!$ 的定义是 $1!=1$ ， $n!=n*(n-1)!$ 。
- 递归函数 $\text{fact}(n)$ 如下。

```
int fact(int n)
{ if (n==1)           //语句1
  return 1;           //语句2
  else                //语句3
    return n*fact(n-1); //语句4
}
```

直接递归函数、
尾递归

- 尾递归算法：可以用循环语句转换为等价的非递归算法
- 其他递归算法：可以通过栈来转换为等价的非递归算法

5.1.1 递归的定义

间接递归示例：

```
void f1(...)  
{  
    ...  
    f2(...);  
    ...  
}
```

```
void f2(...)  
{  
    ...  
    f1(...);  
    ...  
}
```

```
int A(int n) {  
    if (n <= 0) return 0;  
    return B(n - 1);  
}
```

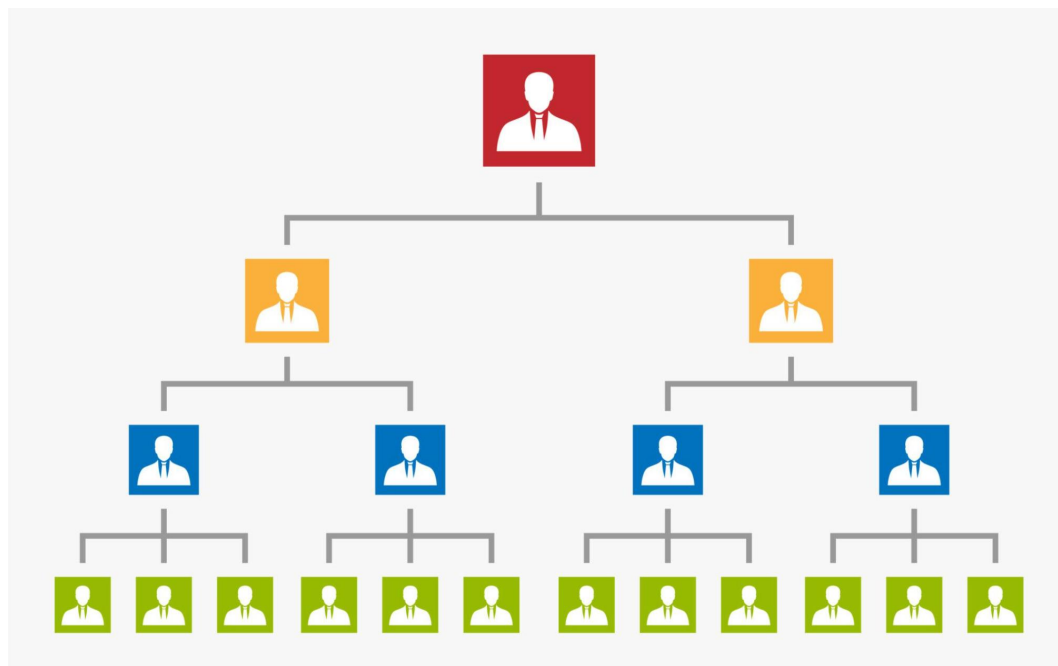
```
int B(int n) {  
    if (n <= 0) return 0;  
    return A(n - 1);  
}
```



```
int A(int n) {  
    if (n <= 1) return 0;  
    return A(n - 2);  
}
```

间接总可以转换为直接递归函数

生活中的递归



找一个人的后代

找某人后代 = 找他的孩子
+
找孩子的后代



打开一个娃娃 → 里面还有一个更小的娃娃
再打开 → 还有更小的
一直到最小那个打不开

5.1.2 何时使用递归

在以下三种情况下，常常要用到递归的方法。

1、定义是递归的

有许多数学公式、数列等的定义是递归的。

例如，求 $n!$ 和Fibonacci数列等。这些问题的求解过程可以将其递归定义直接转化为对应的递归算法。

斐波那契数列以如下被以递推的方法定义：

$$F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2) \quad (n \geq 2, n \in \mathbb{N}^*)$$

5.1.2 何时使用递归

在以下三种情况下，常常要用到递归的方法。

1、定义是递归的

求Fibonacci数列的递归算法

```
int Fib1(int n)           //求Fibonacci数列的第n项
{ if (n==1 || n==2)
    return(1);
  else
    return(Fib1(n-1)+Fib1(n-2));
}
```

2、数据结构是递归的

有些数据结构是递归的。例如，第2章中介绍过的单链表就是一种递归数据结构，其结点类型定义如下：

```
typedef struct LNode
{   ElemType data;
    struct LNode *next;
} LinkNode;
```

指向同类型结点的
指针

递归数据结构

3、问题的求解方法是递归的

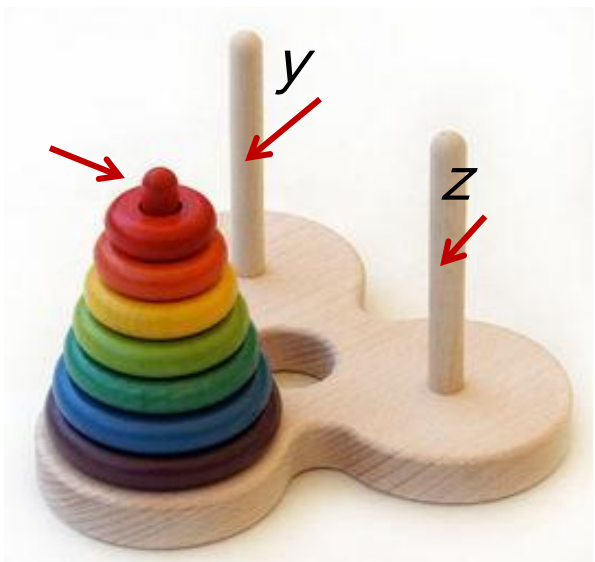
Hanoi问题： X 、 Y 和 Z 的塔座，在塔座 X 上有 n 个直径各不相同，从小到大依次编号为 $1 \sim n$ 的盘片。要求将 X 塔座上的 n 个盘片移到塔座 Z 上。

移动规则：

- 每次只能移动一个盘片；
- 盘片可以插在 X 、 Y 和 Z 中任一塔座上；
- 任何时候都不能将一个较大的盘片放在较小的盘片上方。



- 设 $\text{Hanoi}(n, x, y, z)$ 表示将 n 个盘片从 x 通过 y 移动到 z 上。



$\text{Hanoi}(n, x, y, z)$



```
Hanoi(n-1, x, z, y);  
move(n, x, z):将第n个圆盘从x移到z;  
Hanoi(n-1, y, x, z)
```

“大问题”转化为若干个“小问题”求解

在Hanoi问题的递归算法中，当移动6个盘片时递归次数是多少？

$$t(n) = 1 \quad \text{当 } n=1$$

$$t(n) = 2t(n-1)+1 \quad \text{当 } n>1$$


$$\begin{aligned} t(6) &= 2t(5) + 1 \\ &= 2^2t(4) + 1 + 2 \\ &= 2^3t(3) + 1 + 2 + 2^2 \\ &= 2^4t(2) + 1 + 2 + 2^2 + 2^3 \\ &= 2^5t(1) + 1 + 2 + 2^2 + 2^3 + 2^4 \\ &= 1 + 2 + 2^2 + 2^3 + 2^4 + 2^5 = 2^6 - 1 = 63 \end{aligned}$$

5.1.3 递归模型

递归模型是递归算法的抽象，它反映一个递归问题的递归结构。

例如求 $n!$ 递归算法对应的递归模型如下：

$\text{fun}(1)=1$ (1)

递归出口

$\text{fun}(n)=n*\text{fun}(n-1) \quad n>1$ (2)

递归体

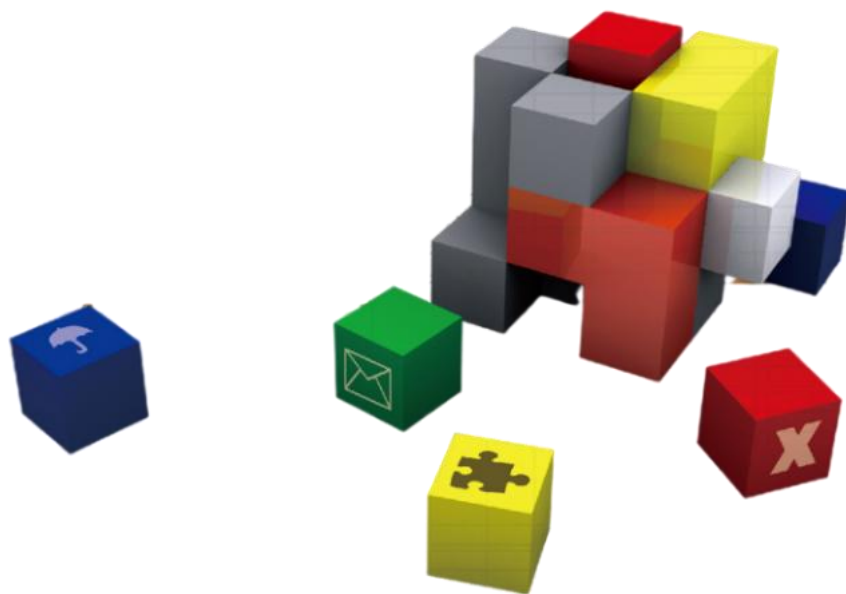
一般地，一个递归模型是由**递归出口**和**递归体**两部分组成。

- **递归出口**确定递归到何时结束。
- **递归体**确定递归求解时的递推关系。

递归出口的一般格式如下：

$$f(s_1) = m_1$$

这里的 s_1 与 m_1 均为常量，有些递归问题可能有几个递归出口。



递归思路

把一个不能或不好直接求解的“**大问题**”转化成一個或几个“**小问题**”来解决；

再把这些“**小问题**”进一步分解成更小的“**小问题**”来解决。



每个“小问题”都可以直接解决（此时分解到递归出口）

但递归分解不是随意的分解，递归分解要**保证**“**大问题**”与“**小问题**”**相似**，即求解过程与环境都相似。

例如，统计全国GDP

大问题



国家统计局 (GDP)

小问题

海南统计局 (GDP)

北京统计局 (GDP)

...

海口统计局 (GDP)

...

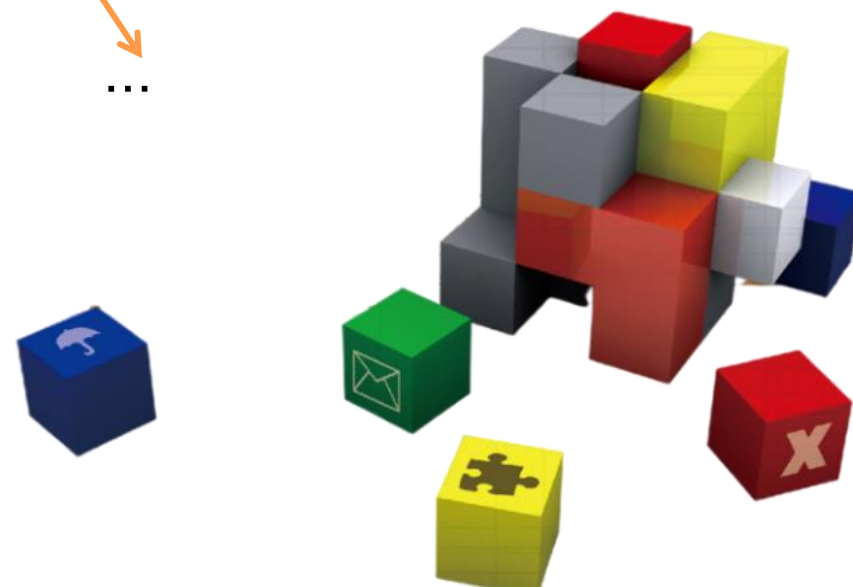
...

美兰区统计局 (GDP)

...

某企业 (GDP)

递归出口



为了讨论方便，简化上述递归模型为：

$$f(s_1)=m_1$$

$$f(s_n)=g(f(s_{n-1}), c_{n-1})$$

求 $f(s_n)$ 的分解过程如下：

$$f(s_n)$$



$$f(s_{n-1})$$



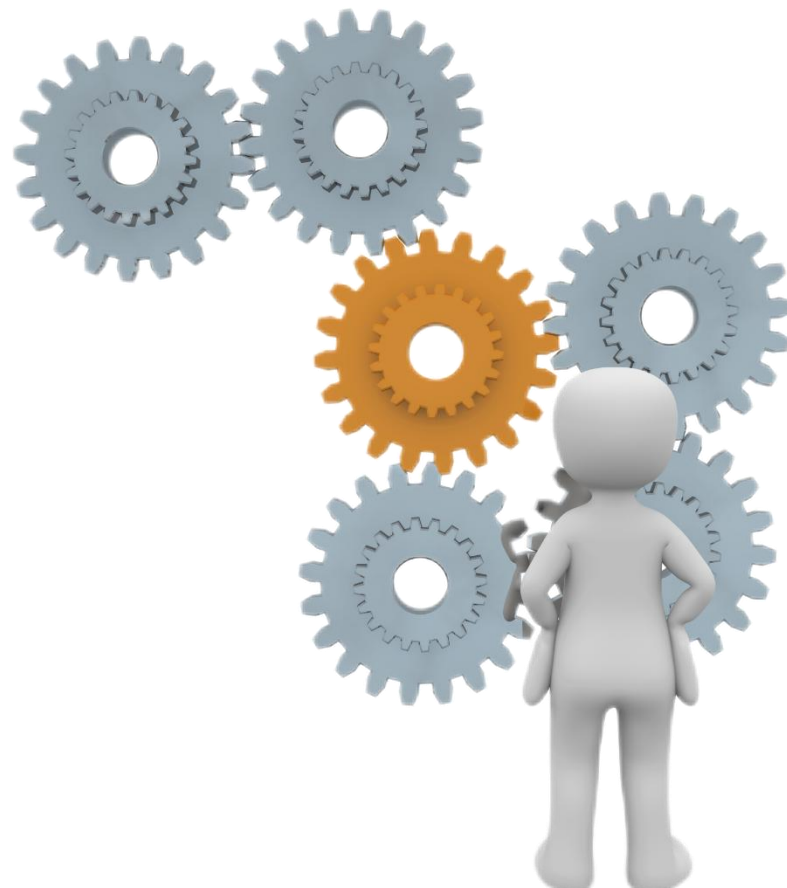
...



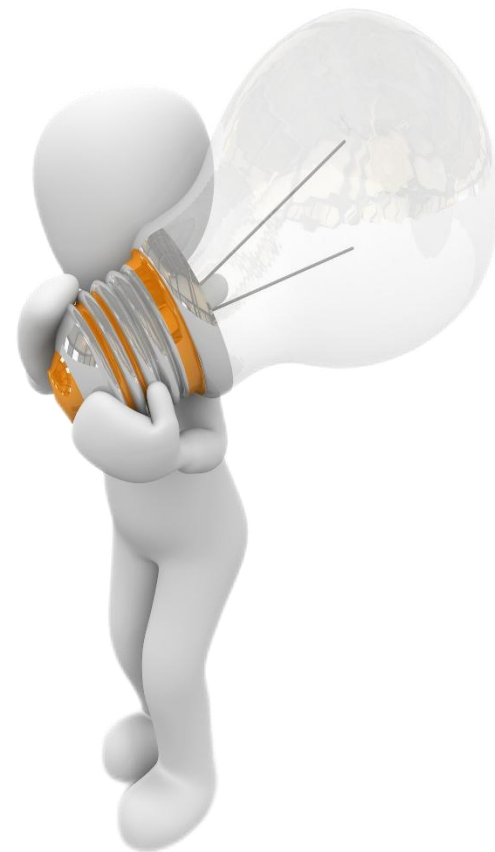
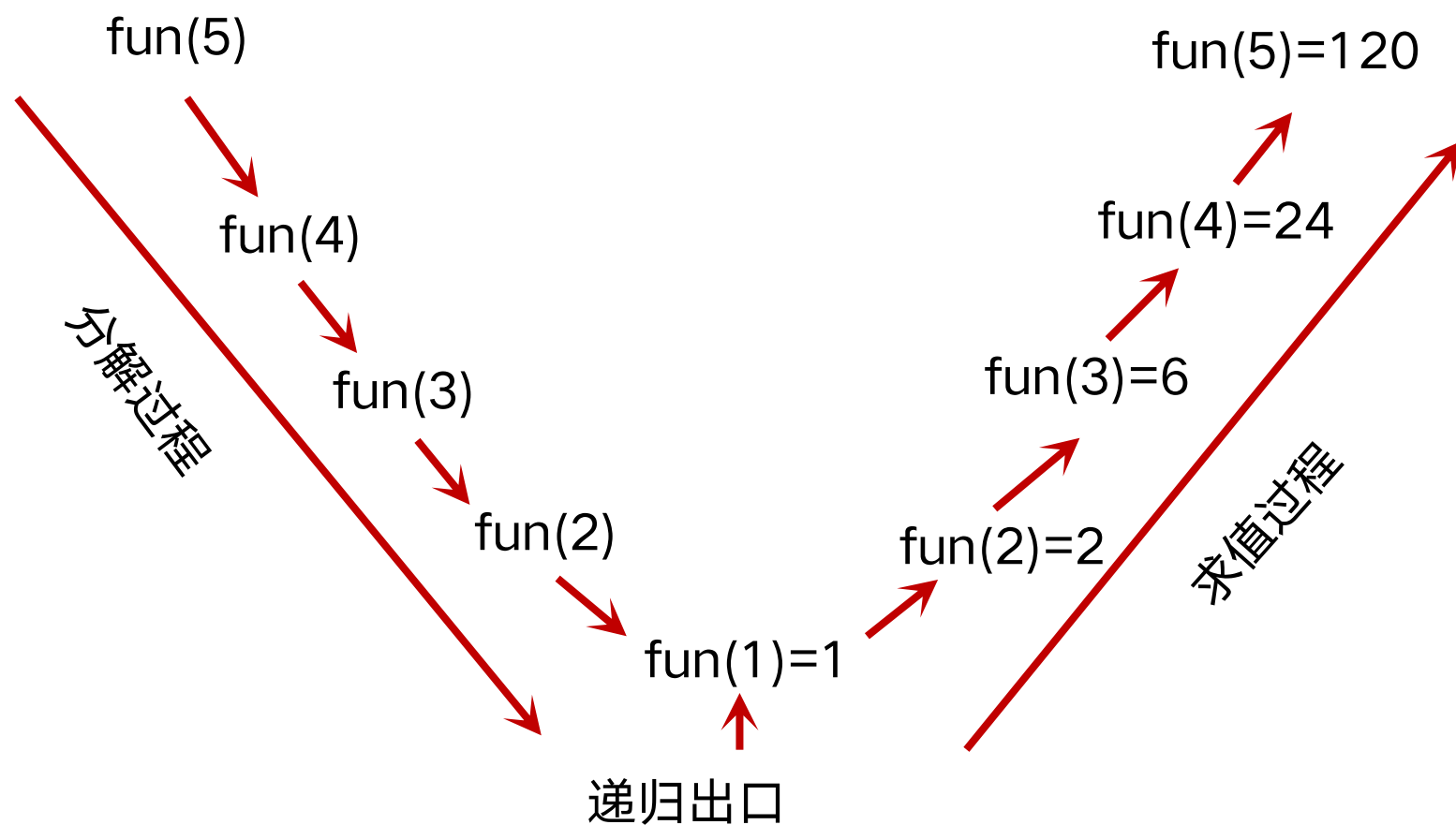
$$f(s_2)$$



$$f(s_1)$$



求解 $\text{fun}(5)$ 即 $5!$ 的过程如下：



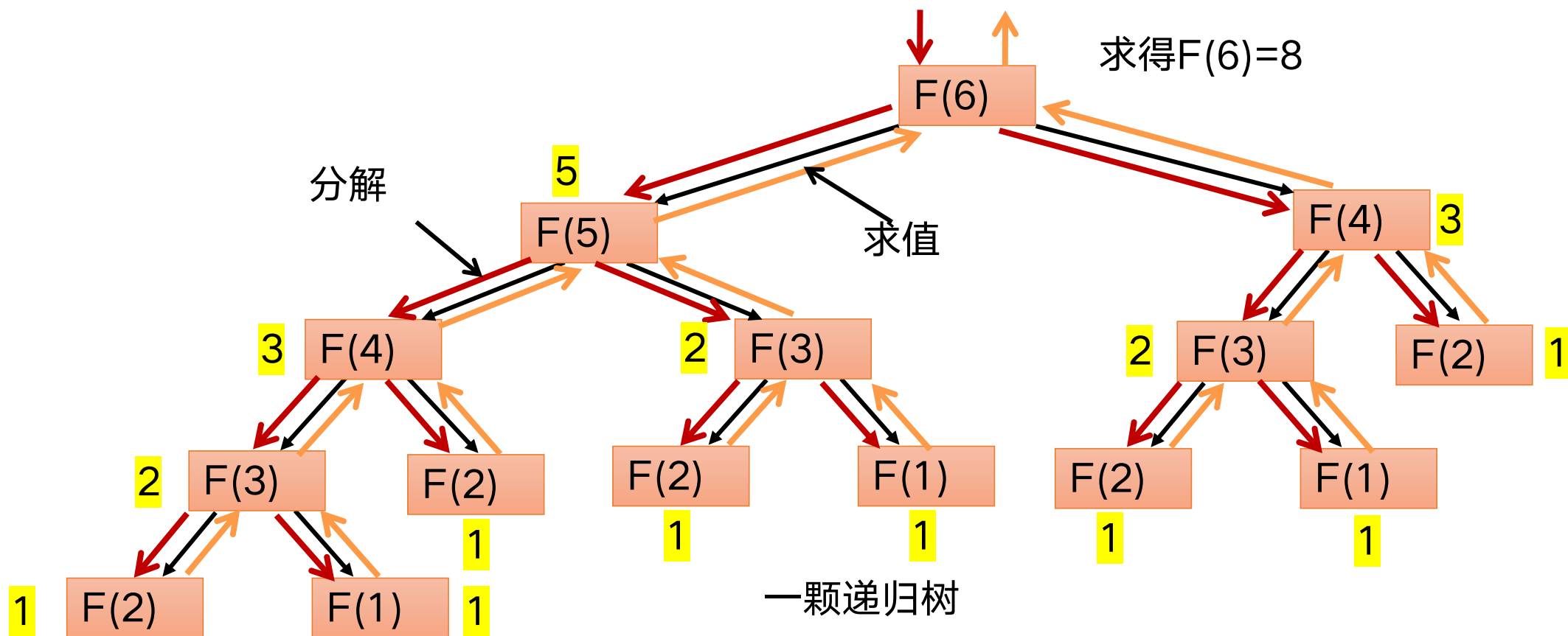
对于复杂的递归问题，在求解时需要进行多次分解和求值。

例如：

$$F(1)=1, F(2)=1$$

$$F(n)=F(n-1)+F(n-2) \quad n>2$$

求 $F(6) = ?$



5.1.4 递归与数学归纳法

- 从递归体看到，如果已知 s_i 、 s_{i+1} 、 $\dots$ 、 s_{n-1} ，就可以确定 s_n 。
- 从数学归纳法的角度来看，这相当于数学归纳法归纳步骤的内容。
- 还应给出这个数列的初始值 s_1 ，对应递归出口。

例如，采用数学归纳法证明下式：

$$1+2+\cdots+n=n(n+1)/2$$

- ❶ 当 $n=1$ 时，左式=1，右式= $1*(1+1)/2=1$ ，左右两式相等，等式成立。
- ❷ 假设当 $n=k-1$ 时等式成立，有 $1+2+\cdots+(k-1)=k(k-1)/2$ 。
- ❸ 当 $n=k$ 时，左式= $1+2+\cdots+k=1+2+\cdots+(k-1)+k=k(k-1)/2+k=k(k+1)/2$
等式成立。即证。

利用数学归纳法证明命题分为两个步骤：

- ❶ 证明当 n 取第一个值 n_0 （ $n_0=0$ 或者1等）时结论成立。
- ❷ 假设 $n=k-1$ （ k 属于可数集并且 $k>n_0$ ）时结论成立，证明 $n=k$ 时结论也成立。



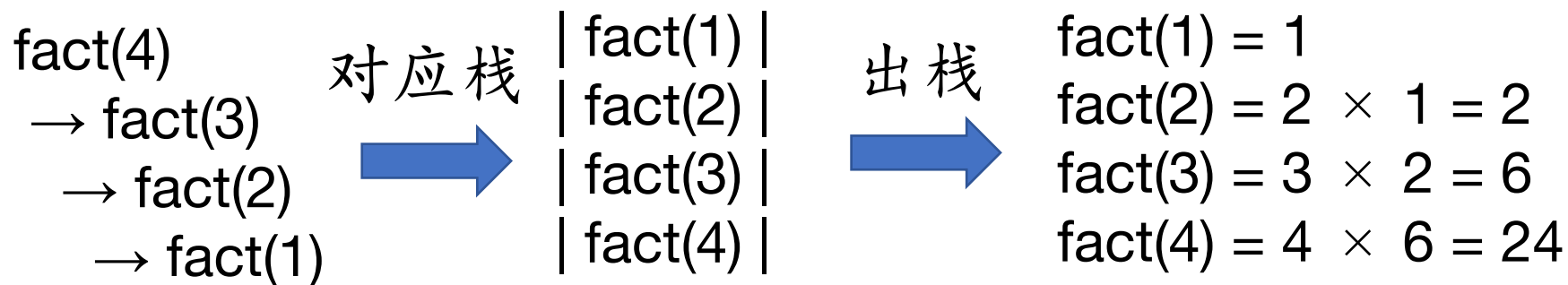
结 论

- 数学归纳法是一种论证方法，而递归是算法和程序设计的一种实现技术。
- 数学归纳法是递归求解问题的理论基础。可以说递归的思想来自数学归纳法。

5.2 栈与递归

递归调用过程中，每一层函数调用都需要保存现场信息，而这些信息按照“后调用先返回”的规律，正好符合栈的后进先出特性，因此递归可以用栈来实现。

```
int fact(int n) {  
    if (n == 1) return 1;  
    return n * fact(n - 1);  
}
```



每调用一次，就压栈

5.2 栈与递归

递归算法如何转换为非递归算法？

- 递归算法可以通过栈转换成非递归形式。
- 对于尾递归，可以用循环递推方法来转换。

第6章 数组和广义表

提纲

CONTENTS



6.1 数 组

6.2 稀疏矩阵

6.3 广义表

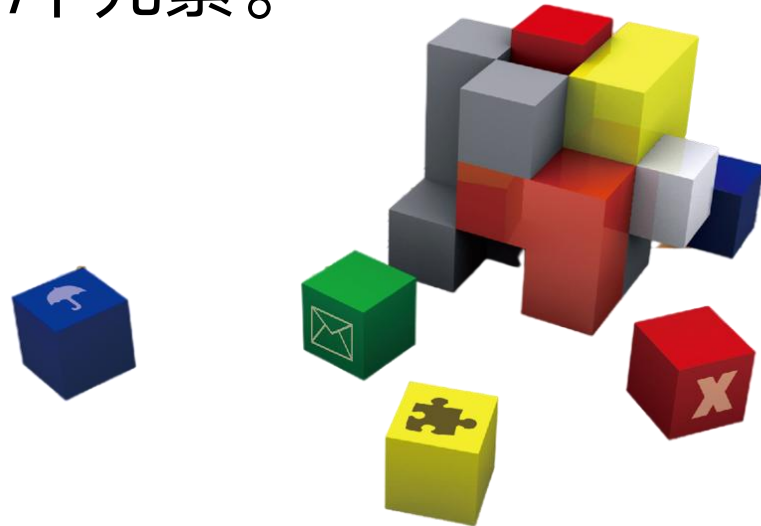
6.1 数 组

6.1.1 数组的基本概念

从逻辑结构上看，一维数组 A 是 n ($n > 1$) 个相同类型数据元素 a_1 、 a_2 、 $\dots$ 、 a_n 构成的有限序列，其逻辑表示为：

$$A = (a_1, a_2, \dots, a_n)$$

其中， a_i ($1 \leq i \leq n$) 表示数组 A 的第 i 个元素。



6.1 数 组

一个m行n列的二维数组A可以看作是每个数据元素都是相同类型的一维数组的一维数组。

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \dots & & & \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix} \quad \longrightarrow \quad A = [A_1, A_2, \dots, A_m]$$



$$A_1 = [a_{1,1}, a_{1,2}, \dots, a_{1,n}]$$

$$A_2 = [a_{2,1}, a_{2,2}, \dots, a_{2,n}]$$

.....

$$A_m = [a_{m,1}, a_{m,2}, \dots, a_{m,n}]$$

➤ 由此看出，多维数组是线性表的推广。

6.1 数 组

6.1.1 数组的基本概念

- 几乎所有的计算机高级语言都实现了数组数据结构，并称之为**数组类型**。
- 以C/C++语言为例，其中数组数据类型具有以下性质：
 - 数组中的**数据元素数目固定**。一旦定义了一个数组，其数据元素数目不再有增减的变化。
 - 数组中的数据元素具有**相同的数据类型**。
 - 数组中的每个数据元素都和一组**唯一的下标**对应。
 - 数组具有随机存取特性。可随机存取数组中的任意数据元素。

6.1 数 组

6.1.1 数组的基本概念

【例6.1】利用数组求解约瑟夫问题：设有 n 个人站成一圈，其编号从 $1 \sim n$ 。从编号为1的人开始按顺时针方向“1, 2, 3, 4, ...”循环报数，数到 m 的人出列，然后从出列者的下一个人重新开始报数，数到 m 的人又出列，如此重复进行，直到 n 个人都出列为止。要求输出这 n 个人的出列顺序。

例如，有8个人的初始序列为

1 2 3 4 5 6 7 8

当 $m=4$ 时，出列顺序为

4 8 5 2 1 3 7 6



6.1 数 组

用一维数组 $p[0..n-1]$ 存放 n 个人 $(1, 2, \dots, n)$

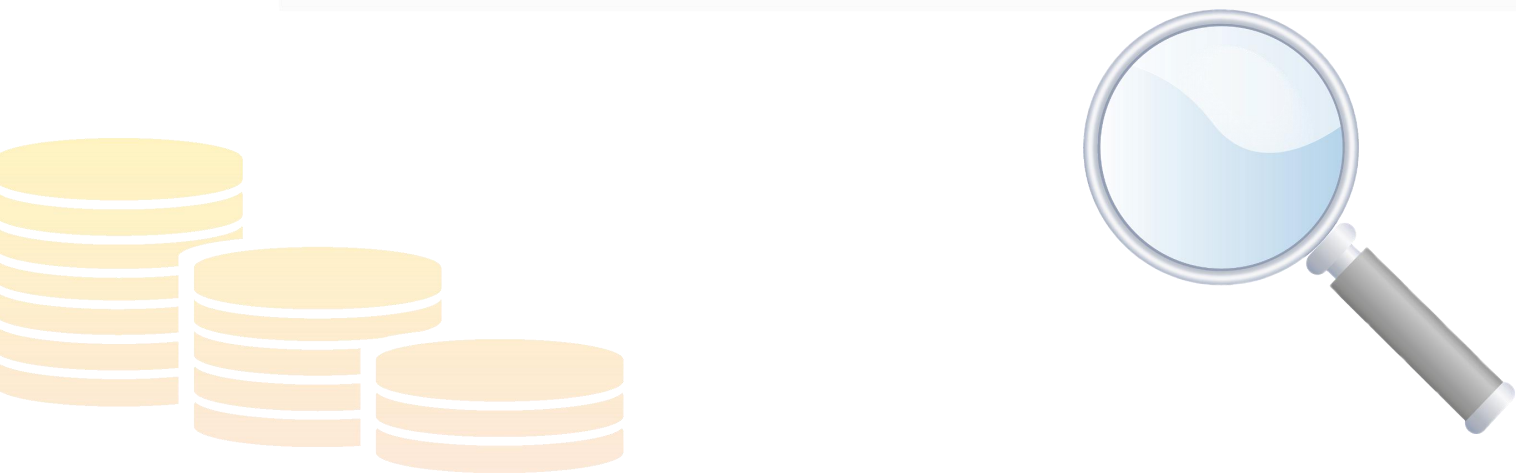
```
void josephus(int n,int m)
{ int p[MaxSize];
  int i,j,t;
  for (i=0;i<n;i++)          //构建初始序列 (1, 2, ..., n)
    p[i]=i+1;
  t=0;                        //首次报数的起始位置
  printf("出列顺序:");
  for (i=n;i>=1;i--)          //i为数组p中当前的人数，出列一次，人数减1
  { t=(t+m-1)%i;               //t为出列者的编号
    printf("%d ",p[t]);        //编号为t的元素出列
    for (j=t+1;j<=i-1;j++)    //后面的元素前移一个位置
      p[j-1]=p[j];
  }
  printf("\n");
}
```

6.1 数 组

6.1.1 数组的基本概念

注 意：

- 本章的数组是作为一种数据结构讨论的。
- 而C/C++中的数组是一种数据类型。
- 前者可以借助后者来存储，像线性表的顺序存储结构即顺序表就是借助一维数组这种数据类型来存储的。但两者**不能混淆**。



6.1 数 组

6.1.2 数组的存储结构

数组特别适合采用顺序存储结构 $\Rightarrow$ 将数组所有元素存储在一块地址连续的内存单元中。

- 数组不能采用链式存储结构吗？
 - 为什么数组一般采用顺序存储结构？
-
- 因为数组这个数据结构主要有存取两种操作，利用顺序存储结构更加高效
 - 存储密度较高



6.1 数 组

6.1.2 数组的存储结构

1、一维数组的存储结构

$$A = (a_1, a_2, \dots, a_i, \dots, a_n)$$

一旦 a_1 的存储地址 $LOC(a_1)$ 确定，并假设每个数据元素占用 k 个存储单元，则任一数据元素 a_i 的存储地址 $LOC(a_i)$ 就可由以下公式求出：

$$LOC(a_i) = LOC(a_1) + (i-1) * k \quad (0 \leq i \leq n)$$

共 $i-1$ 个元素

数组 a : $a_1 \quad a_2 \quad a_3 \quad \dots \quad a_{i-1} \quad a_i \quad \dots \quad a_n$



一维数组具有**随机存储特性**：可以在 $O(1)$ 时间内找到序号为 i 的元素值。

6.1 数 组

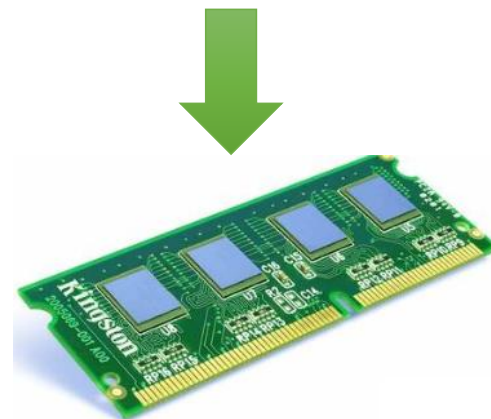
6.1.2 数组的存储结构

2、二维数组的存储结构

m 行 n 列的二维数组 $A_{m \times n}$ ，存储方式：

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \dots & \dots & \dots & \dots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix}$$

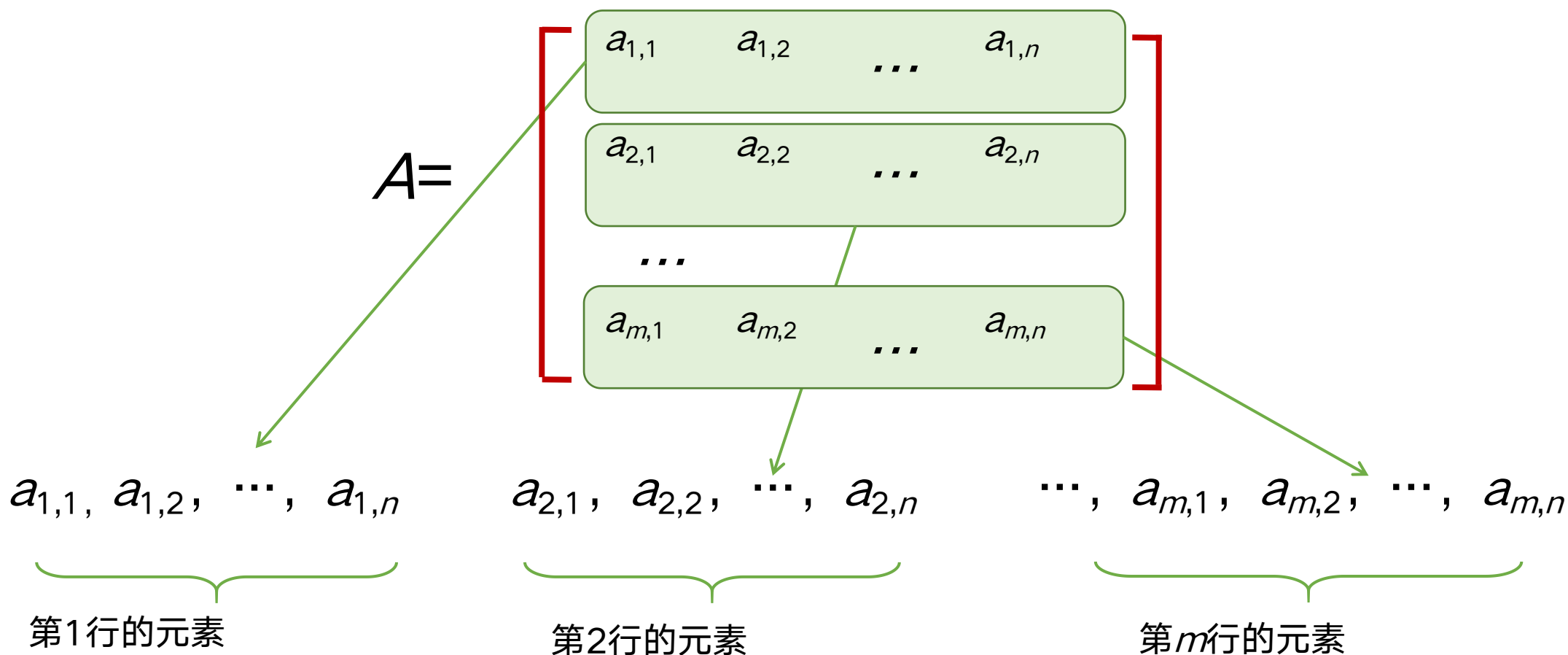
- 以行序为主序的存储
- 以列序为主序的存储



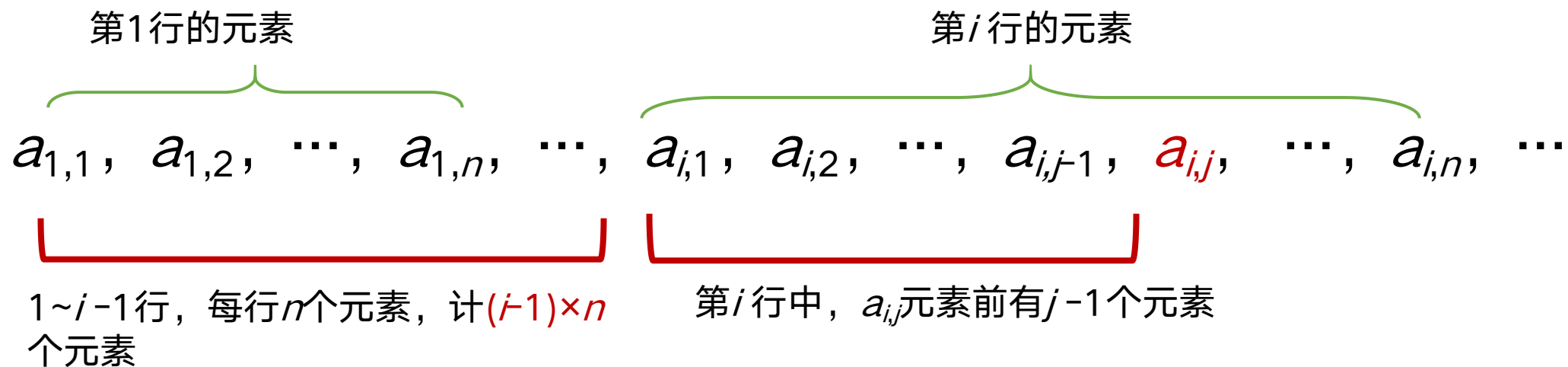
6.1 数 组

6.1.2 数组的存储结构

① 以行序为主序的存储方式



6.1 数组



则 $a_{i,j}$ 元素前共有 $(i-1) \times n + j - 1$ 个元素

$$\text{LOC}(a_{i,j}) = \text{LOC}(a_{1,1}) + [(i-1) \times n + (j-1)] \times k$$

6.1 数组

6.1.2 数组的存储结构

② 以列序为主序的存储方式

同理可推出在**以列序为主序**的计算机系统中有)：

$$\text{LOC}(a_{i,j}) = \text{LOC}(a_{1,1}) + [(j-1) \times m + (i-1)] \times k$$



二维数组采用顺序存储结构时，也具有**随机存取特性**。



是指给定序号 (i,j) ，可以在 $O(1)$ 的时间内找到相应的元素值。



多维数组采用顺序存储时具有随机存取特性。

6.1 数 组

6.1.3 特殊矩阵的压缩存储

特殊矩阵的主要形式有：

- (1) 对称矩阵
- (2) 上三角矩阵 / 下三角矩阵
- (3) 对角矩阵

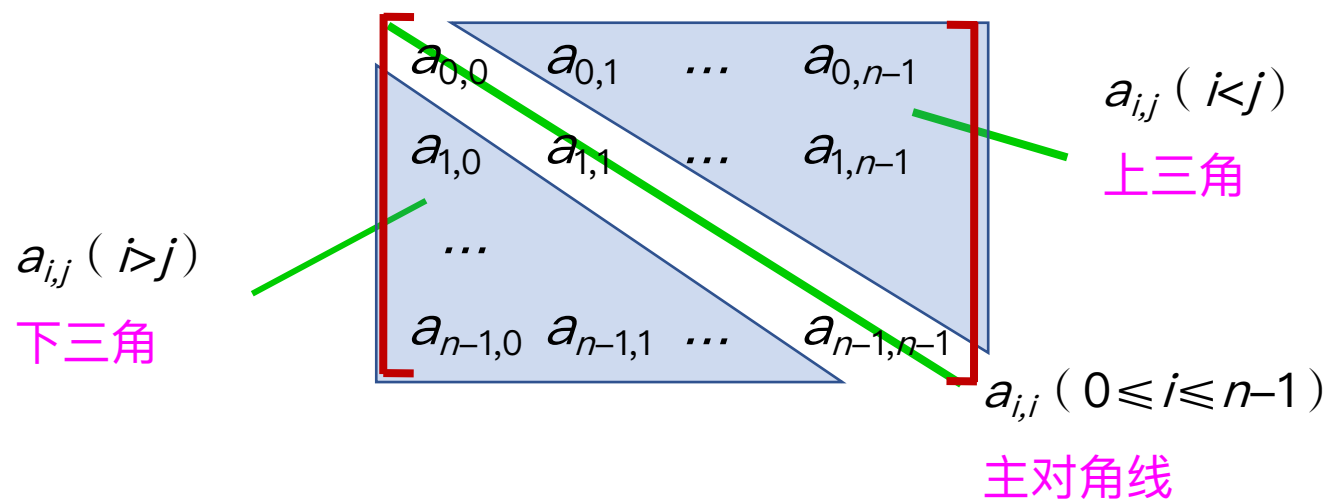
它们都是方阵，即行数和列数相同。



6.1 数 组

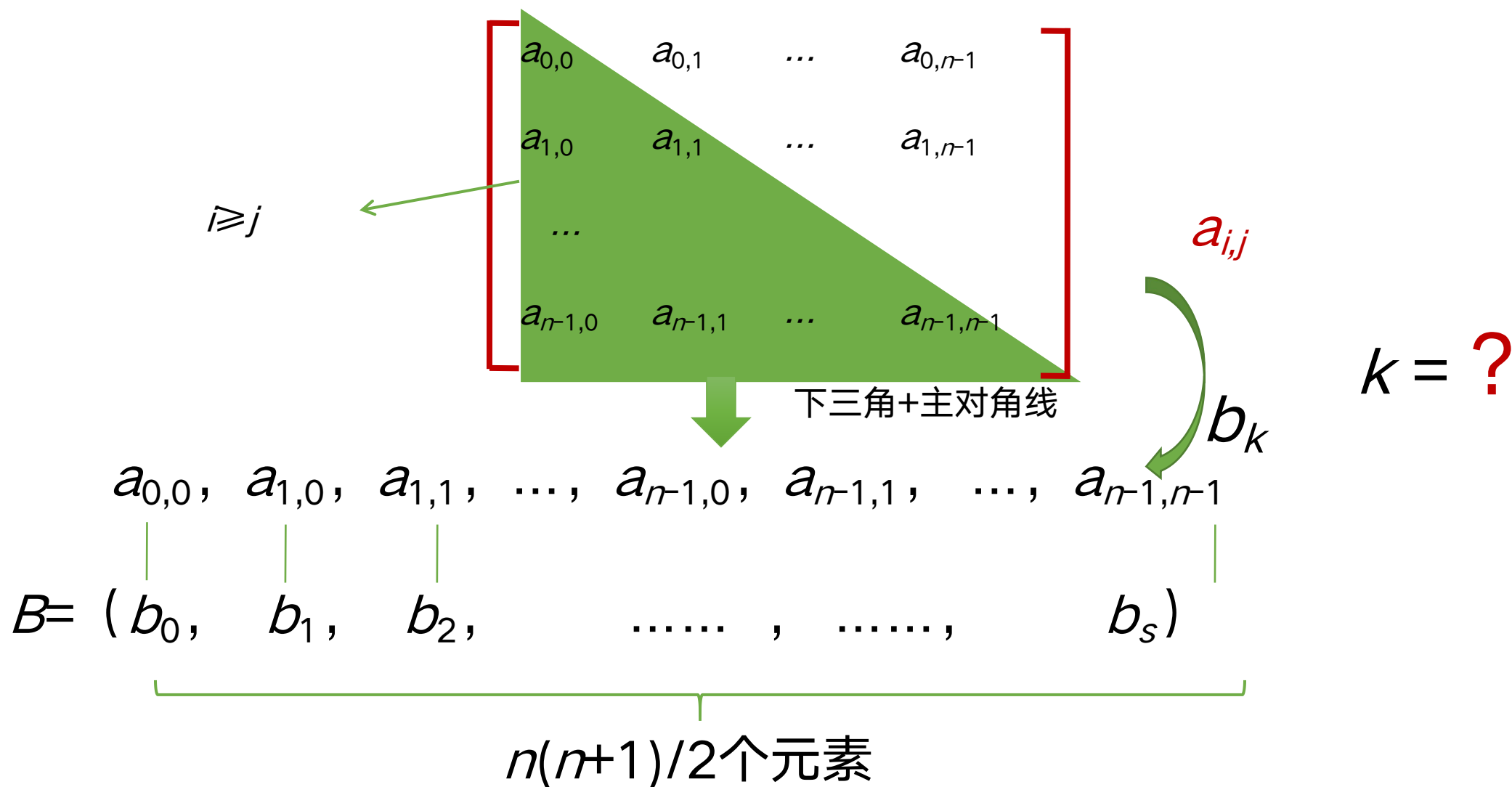
6.1.3 对称矩阵的压缩存储

若一个 n 阶方阵 $A[n][n]$ 中的元素满足 $a_{i,j}=a_{j,i}$ ($0 \leq i, j \leq n-1$)，则称其为 n 阶**对称矩阵**。



6.1 数 组

以行序为主序存储其下三角+主对角线的元素。



6.1 数组

$$B = (a_{0,0}, a_{1,0}, a_{1,1}, \dots, a_{i-1,0}, \dots, a_{i-1,i-1}, a_{i,0}, \dots, a_{i,j-1}, a_{i,j}, \dots, a_{n-1,n-1})$$

$\underbrace{\hspace{1.5cm}}$ $\underbrace{\hspace{1.5cm}}$ $\underbrace{\hspace{2.5cm}}$ $\underbrace{\hspace{1.5cm}}$

$i=0$ 时: 1个元素 $i=1$ 时: 2个元素 $i=i-1$ 时: i 个元素 $i=i$ 时: j 个元素

$\underbrace{\hspace{15cm}}$

共计 $i(i+1)/2 + j$ 个元素

b_k

$\updownarrow$

$$k = \begin{cases} \frac{i(i+1)}{2} + j & \text{当 } i \geq j \text{ 时 (下三角+主对角线的元素)} \\ \frac{j(j+1)}{2} + i & \text{当 } i < j \text{ 时 (} a_{i,j} = a_{j,i} \text{)}$$

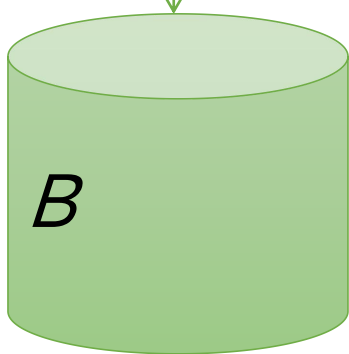
6.1 数 组

6.1.3 特殊矩阵的压缩存储

对于对称矩阵 A ，采用一维数组 B 存储，并提供 A 的所有运算。

n 阶对称矩阵 $A[n][n]$

压缩存储



■ 存元素 $A[i][j]=x$: $k=f(i,j)$, $B[k]=x$

■ 取元素 $y=A[i][j]$: $k=f(i,j)$, $y=B[k]$

提供类似 $A[i][j]$ 的操作

6.1 数 组

6.1.3 特殊矩阵的压缩存储

对n阶对称矩阵压缩存储，需要表长为 () 的顺序表

A. $n/2$

B. $n*n/2$

C. $n*(n+1)/2$

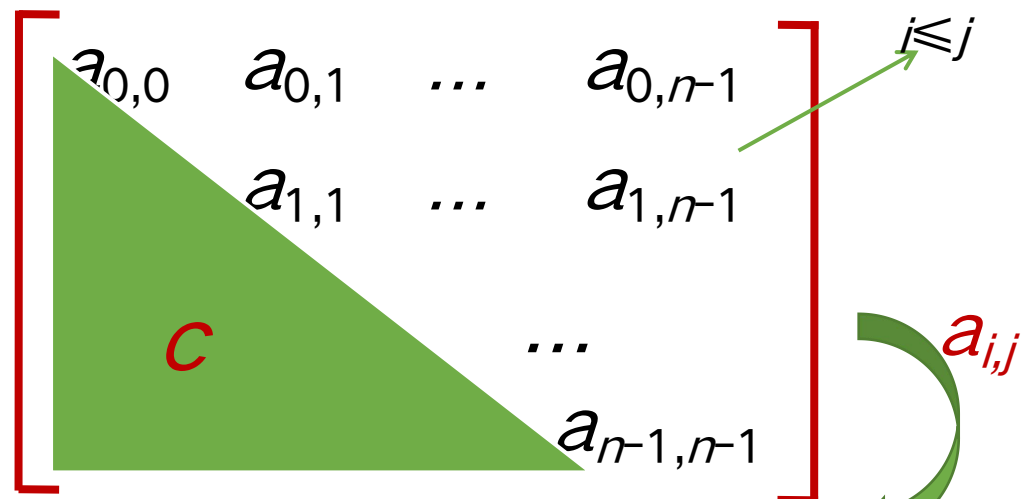
D. $n*(n-1)/2$

答案：C

6.1 数 组

② 三角矩阵的压缩存储

上三角矩阵



$B = (a_{0,0}, a_{0,1}, \dots, a_{0,n-1}, a_{1,1}, \dots, a_{i-1,n-1},$

$\dots,$

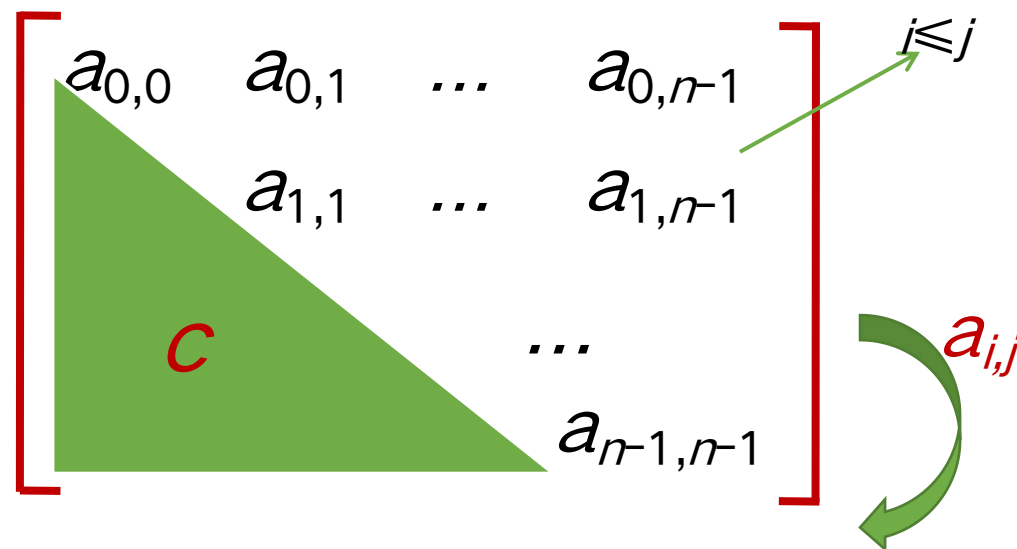
$a_{i-1,i-1}, \dots, a_{i-1,n-1},$

$a_{i,i}, \dots, a_{i,j-1}, a_{i,j}, \dots)$

b_k

6.1 数组

$B =$



$(a_{0,0}, a_{0,1}, \dots, a_{0,n-1}, a_{1,1}, \dots, a_{1,n-1}, \dots, a_{i-1,i-1}, \dots, a_{i-1,n-1}, a_{i,i}, \dots, a_{i,j-1}, a_{i,j}, \dots)$

n 个元素

$n-1$ 个元素

$n-i+1$ 个元素

$j-i$ 个元素

共计 $i(2n-i+1)/2 + j-i$ 个元素

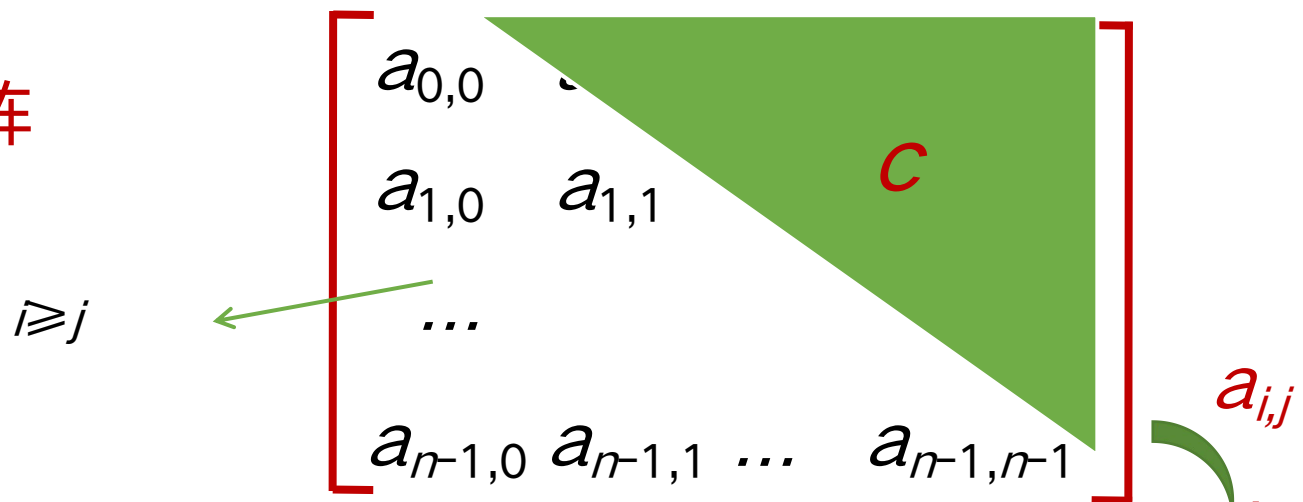
$$k = \begin{cases} \frac{i(2n-i+1)}{2} + j - i & \text{当 } i \leq j \text{ 时} \\ \frac{n(n+1)}{2} & \text{当 } i > j \text{ 时} \end{cases}$$

存放常量 c

6.1 数组

6.1.3 特殊矩阵的压缩存储

下三角矩阵



$$B = (a_{0,0}, a_{1,0}, a_{1,1}, \dots, a_{n-1,0}, a_{n-1,1}, \dots, a_{n-1,n-1})$$

$$k = \begin{cases} \frac{i(i+1)}{2} + j & \text{当 } i \geq j \text{ 时} \\ \frac{n(n+1)}{2} & \text{当 } i < j \text{ 时} \end{cases}$$

存放一个常量 c

6.1 数 组

示例

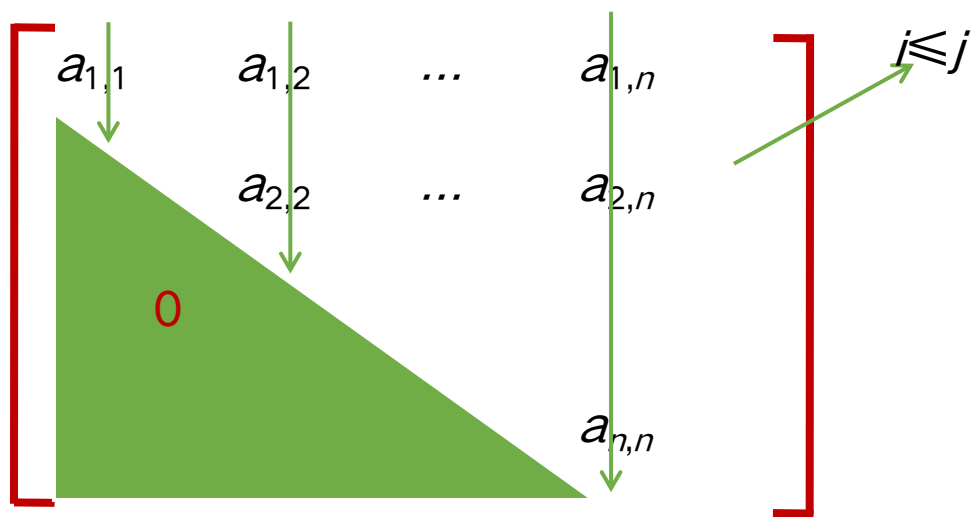
若将 n 阶上三角矩阵 A 按列优先顺序压缩存放在一维数组 $B[1..n(n+1)/2]$ 中， A 中第一个非零元素 $a_{1,1}$ 存于 B 数组的 b_1 中，则应存放 b_k 中的非零元素 $a_{i,j}$ ($i \leq j$) 的下标 i 、 j 与 k 的对应关系是 ()。

A. $i(i+1)/2+j$

B. $i(i-1)/2+j$

C. $j(j+1)/2+i$

D. $j(j-1)/2+i$



1 ~ $j-1$ 列的元素个数: $j(j-1)/2$

第 j 列 a_{ij} 之前的元素个数: $i-1$

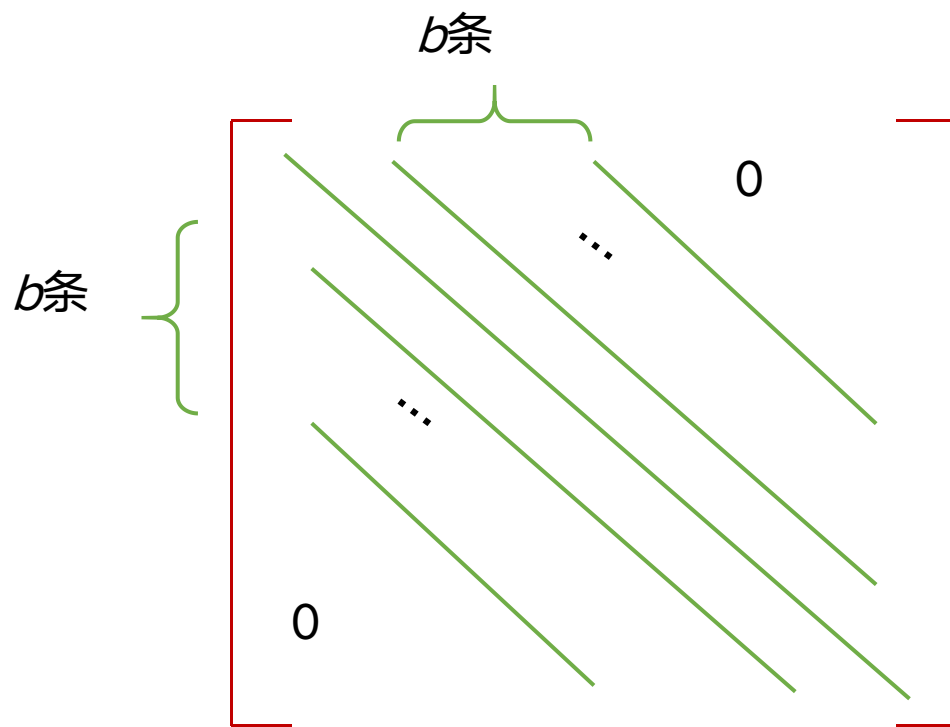
$k = j(j-1)/2 + i - 1 + 1 = j(j-1)/2 + i$

- 按行还是按列
- 初始下标从0还是从1开始

6.1 数 组

6.1.3 特殊矩阵的压缩存储

③ 对角矩阵的压缩存储



半带宽为 b 的对角矩阵

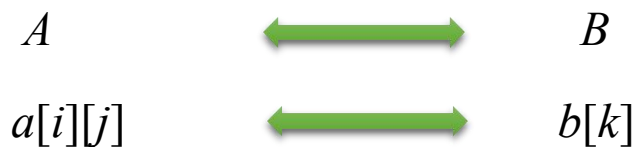


6.1 数 组

6.1.3 特殊矩阵的压缩存储

对角矩阵

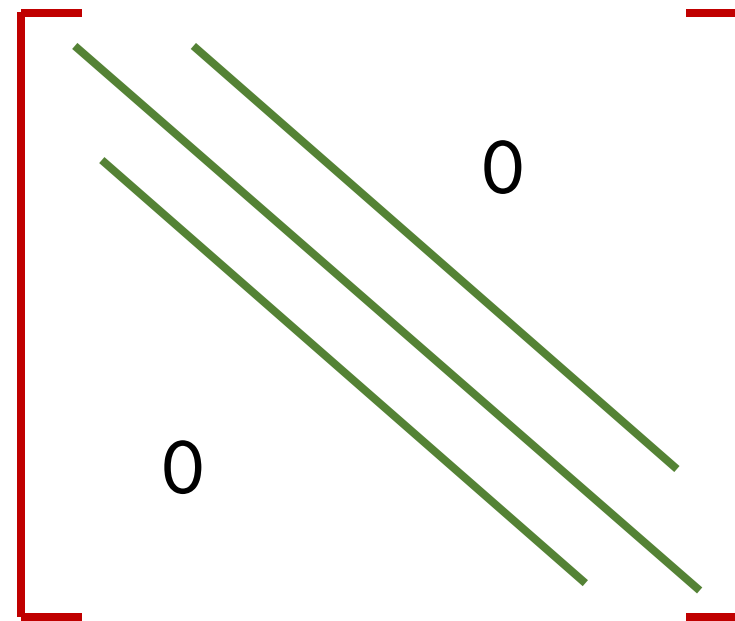
压缩存储



当 $b=1$ 时称为三对角矩阵

其压缩地址计算公式如下：

$$k = 2i + j$$



提纲

CONTENTS

6.1 数 组



6.2 稀疏矩阵

6.3 广义表

6.2 稀疏矩阵

稀疏矩阵的定义

一个阶数较大的矩阵中的非零元素个数 s 相对于矩阵元素的总个数 t 十分小时，即 $s \ll t$ 时，称该矩阵为**稀疏矩阵**。

例如一个 100×100 的矩阵，若其中只有100个非零元素，就可称其为稀疏矩阵。



定性的描述

6.2 稀疏矩阵

稀疏矩阵和特殊矩阵的不同点：

- 特殊矩阵的特殊元素（值相同元素、常量元素）**分布有规律**。
- 稀疏矩阵的特殊元素（非0元素）**分布没有规律**。



6.2 稀疏矩阵

6.2.1 稀疏矩阵的三元组表示

稀疏矩阵的压缩存储方法是只存储非零元素。

稀疏矩阵中的每一个非零元素需由一个三元组： $(i, j, a_{i,j})$

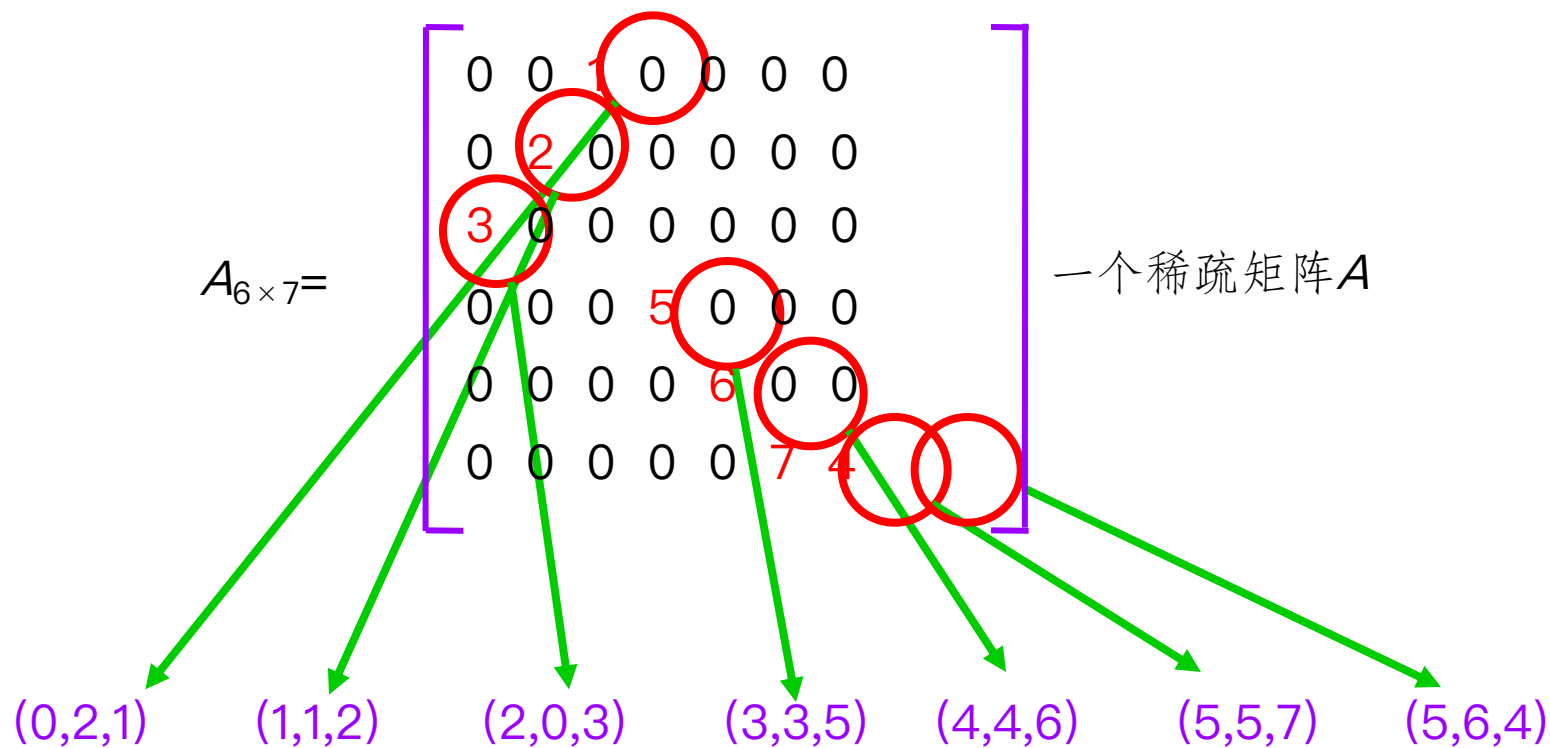
唯一确定，稀疏矩阵中的所有非零元素构成三元组线性表。



6.2 稀疏矩阵

稀疏矩阵三元组表示的演示

一个 6×7 阶稀疏矩阵 A 的三元组线性表表示



三元组线性表:

$((0,2,1), (1,1,2), (2,0,3), (3,3,5), (4,4,6), (5,5,7), (5,6,4))$

6.2 稀疏矩阵

把稀疏矩阵的三元组线性表按顺序存储结构存储，则称为**稀疏矩阵的三元组顺序表**。
类型声明如下：

```
#define MaxSize 100    //矩阵中非零元素最多个数
```

```
typedef struct
```

```
{ int r;           //行号
```

```
  int c;           //列号
```

```
  ElemType d;      //元素值
```

```
} TupNode;         //三元组定义
```

```
typedef struct
```

```
{ int rows;        //行数值
```

```
  int cols;        //列数值
```

```
  int nums;        //非零元素个数
```

```
  TupNode data[MaxSize];
```

```
} TSMatrix;        //三元组顺序表定义
```

存放整个稀疏矩阵

存放一个非0元素

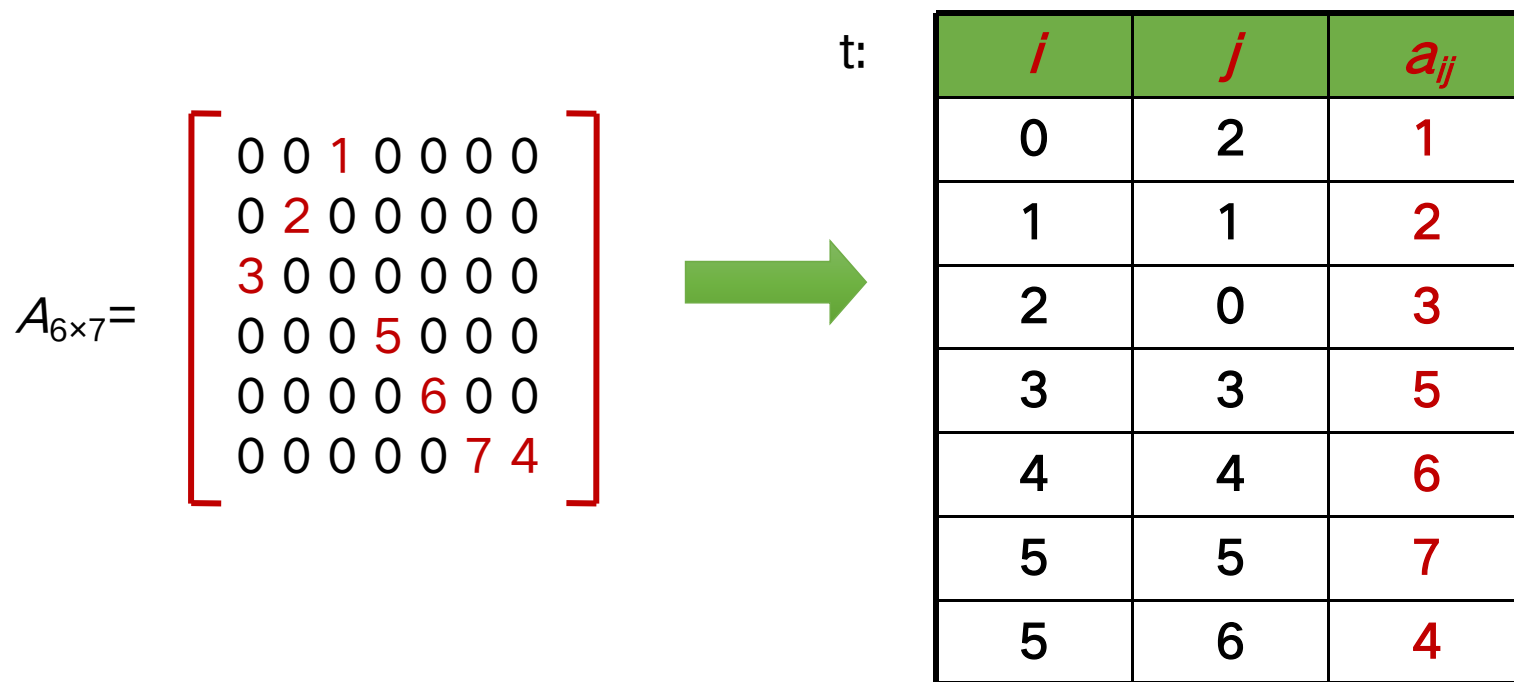
约定：data域中表示的非零元素通常**以行序为主序顺序排列**，它是一种下标按行有序的存储结构。

6.2 稀疏矩阵

稀疏矩阵三元组表示的演示

(1) 从一个二维矩阵创建其三元组表示

以行序方式扫描二维矩阵 A ，将其非零的元素插入到三元组 t 的后面。



6.2 稀疏矩阵

```
void CreatMat(TSMatrix &t, ElemType A[M][N])
{ int i,j; t.rows=M; t.cols=N; t.nums=0;
  for (i=0;i<M;i++)
  { for (j=0;j<N;j++)
    { if (A[i][j]!=0)
      { t.data[t.nums].r=i;
        t.data[t.nums].c=j;
        t.data[t.nums].d=A[i][j];
        t.nums++;
      }
    }
  }
}
```

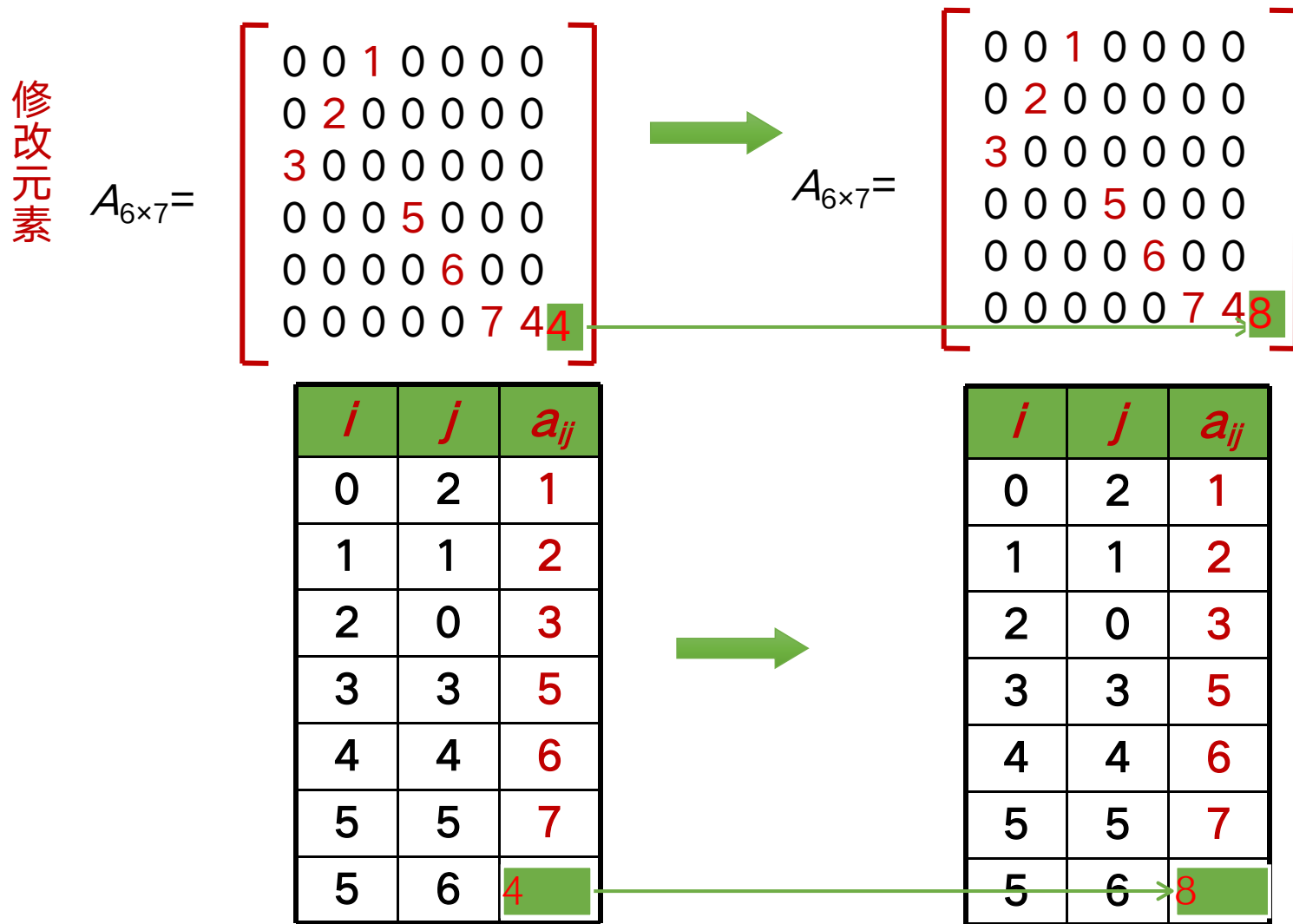
按行序方式扫描所有元素

只存储非零元素

6.2 稀疏矩阵

(2) 三元组元素赋值: $A[i][j]=x$

分为两种情况: ① 将一个非0元素修改为另一个非0值, 如 $A[5][6]=8$ 。



6.2 稀疏矩阵

② 将一个0元素修改为非0值。如 $A[3][5]=8$

增加元素

$$A_{6 \times 7} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 7 & 4 \end{bmatrix}$$



$$A_{6 \times 7} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 8 & 0 \\ 0 & 0 & 0 & 0 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 7 & 4 \end{bmatrix}$$

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4



i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
3	5	8
4	4	6
5	5	7
5	6	4

6.2 稀疏矩阵

算法如下：

```
bool Value(TSMatrix &t, ElemType x, int i, int j)
{
    int k=0, k1;
    if (i>=t.rows || j>=t.cols)
        return false; //失败时返回false

    while (k<t.numr && i>t.data[k].r) k++; //查找行
    while (k<t.numr && i==t.data[k].r
        && j>t.data[k].c) k++; //查找列
}
```

在t中按行、列号查找

6.2 稀疏矩阵

```
if (t.data[k].r==i && t.data[k].c==j) //存在这样的元素
    t.data[k].d=x;

else //不存在这样的元素时插入一个元素
{
    for (k1=t.nums-1;k1>=k;k1--)
    {
        t.data[k1+1].r=t.data[k1].r;
        t.data[k1+1].c=t.data[k1].c;
        t.data[k1+1].d=t.data[k1].d;
    }
    t.data[k].r=i;t.data[k].c=j;t.data[k].d=x;
    t.nums++;
}

return true; //成功时返回true
}
```

修改元素

增加元素

6.2 稀疏矩阵

(3) 将指定位置的元素值赋给变量 执行 $x=A[i][j]$

先在三元组t中找到指定的位置，再将该处的元素值赋给x。

```
bool Assign(TSMatrix t, ElemType &x, int i, int j)
```

```
{ int k=0;
```

```
  if (i>=t.rows || j>=t.cols)
```

```
    return false;
```

//失败时返回false

```
  while (k<t.nums && i>t.data[k].r) k++; //查找行
```

```
  while (k<t.nums && i==t.data[k].r
```

```
    && j>t.data[k].c) k++; //查找列
```

在t中按行、
列号查找

```
  if (t.data[k].r==i && t.data[k].c==j)
```

```
    x=t.data[k].d;
```

```
  else
```

```
    x=0;
```

```
  return true;
```

//成功时返回true

```
}
```

找到了非
0的元素

没有找到:
为0元素

6.2 稀疏矩阵

(4) 输出三元组

从头到尾扫描三元组t，依次输出元素值。

```
void DispMat(TSMatrix t)
{ int i;
  if (t.nums<=0) return;
  printf( "\t%d\t%d\t%d\n",t.rows,t.cols,t.nums);
  printf(" ----- \n");
  for (i=0;i<t.nums;i++)
    printf("\t%d\t%d\t%d\n ",t.data[i].r,t.data[i].c,t.data[i].d);
}
```


6.2 稀疏矩阵

(5) 矩阵转置

对于一个 $m \times n$ 的矩阵 $A_{m \times n}$ ，其转置矩阵是一个 $n \times m$ 的矩阵 $B_{n \times m}$ ，满足 $b_{i,j} = a_{j,i}$ ，其中 $0 \leq i \leq m-1$ ， $0 \leq j \leq n-1$ 。

$$A_{6 \times 7} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 7 & 4 \end{bmatrix}$$

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4



$$B_{7 \times 6} = \begin{bmatrix} 0 & 0 & 3 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 6 & 0 \\ 0 & 0 & 0 & 0 & 0 & 7 \\ 0 & 0 & 0 & 0 & 0 & 4 \end{bmatrix}$$

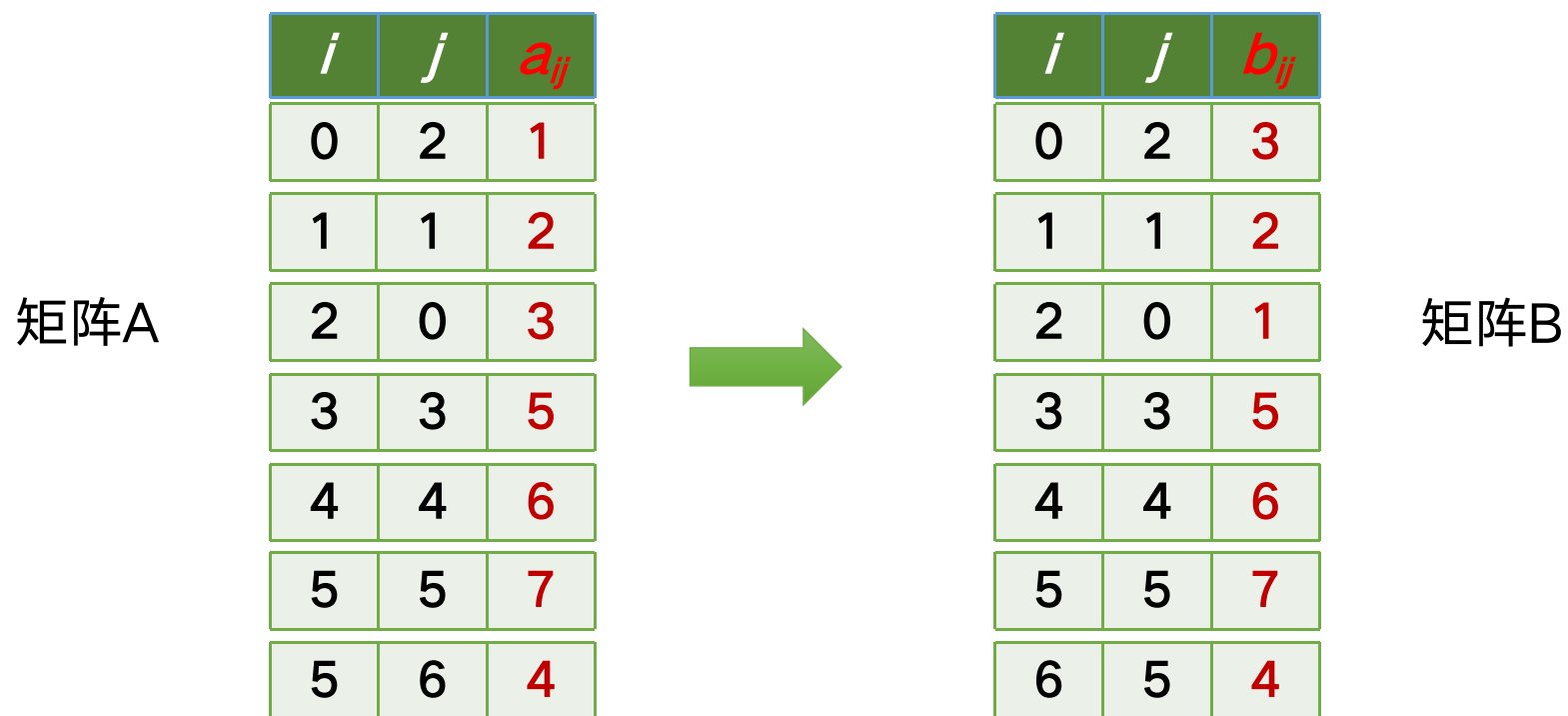
i	j	b_{ij}
0	2	3
1	1	2
2	0	1
3	3	5
4	4	6
5	5	7
6	5	4



6.2 稀疏矩阵

(5) 矩阵转置

一种非高效的算法：按第0、1、2、 $\dots$ 、 $n-1$ 列进行转换



矩阵转置

m 行 n 列， t 个非0元素，时间复杂度为 $O(nt)$ 。

6.2 稀疏矩阵

例题（2018统考）.假设有一个 12×12 的对称矩阵M，将其上三角部分的元素 $m_{i,j}$ （ $1 \leq i \leq j \leq 12$ ）按行存入C语言的一维数组N中（首位为0），元素 $m_{6,6}$ 在N中的下标是：

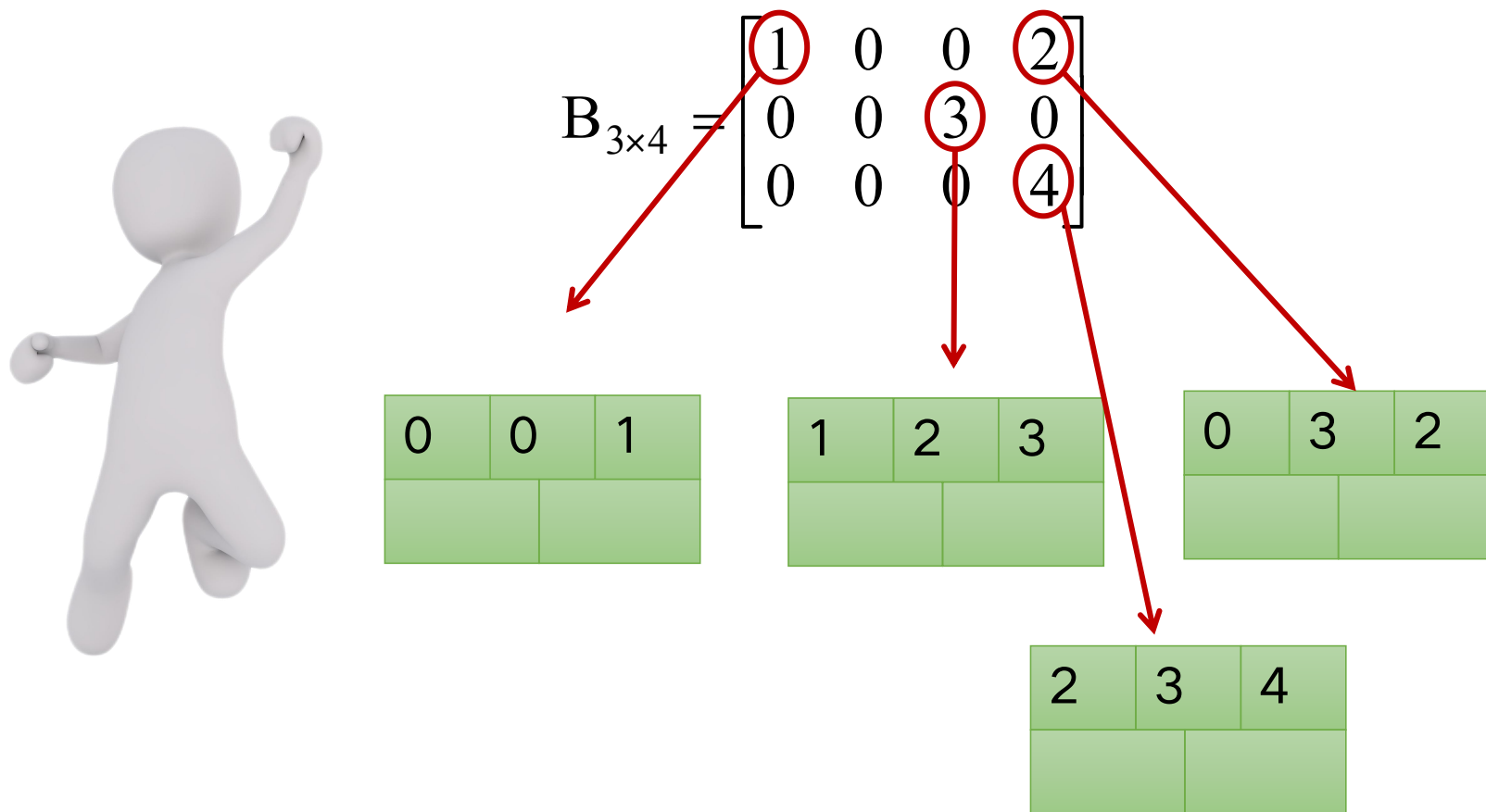
- A.50
- B.51
- C.55
- D.66

答案：A

6.2 稀疏矩阵

6.2.2 稀疏矩阵的十字链表表示

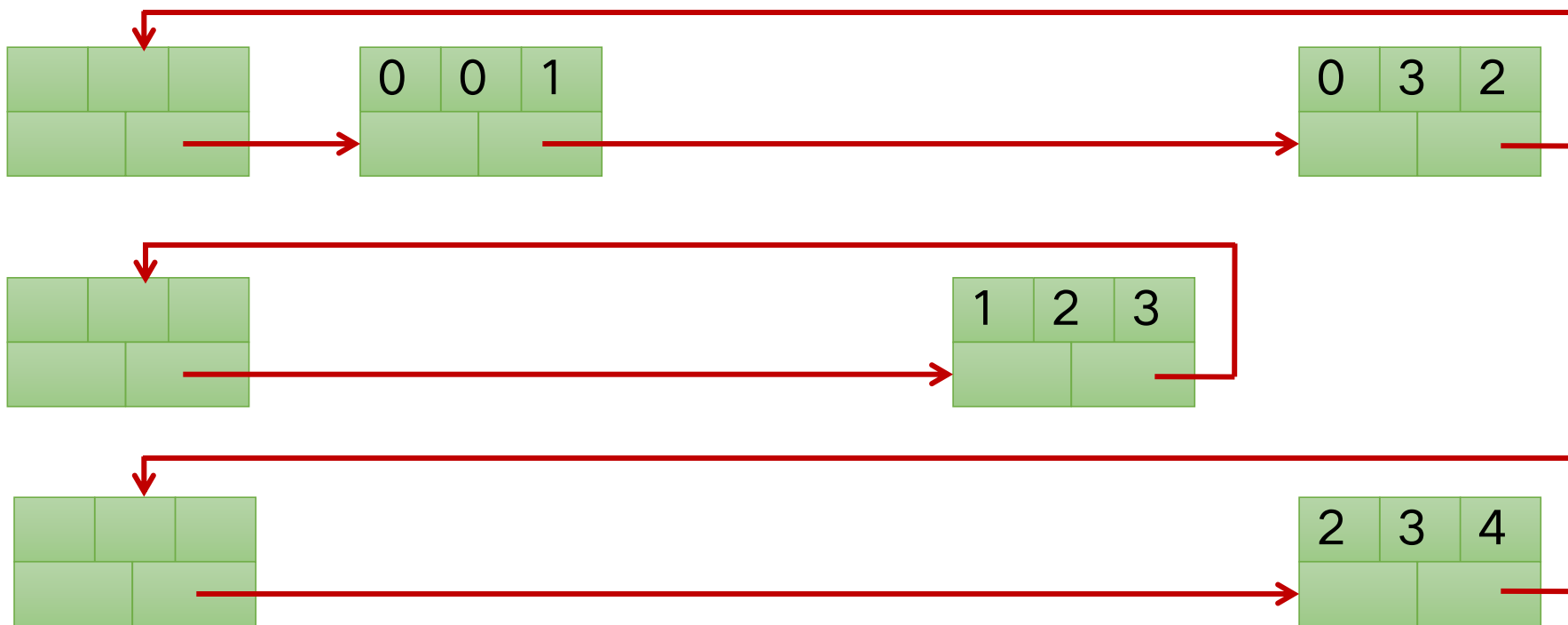
■ 每个非零元素对应一个结点。



6.2 稀疏矩阵

6.2.2 稀疏矩阵的十字链表表示

- 每行的所有结点链起来构成一个带行头结点的循环单链表。以 $h[i]$ ($0 \leq i \leq m-1$) 作为第 i 行的头结点。

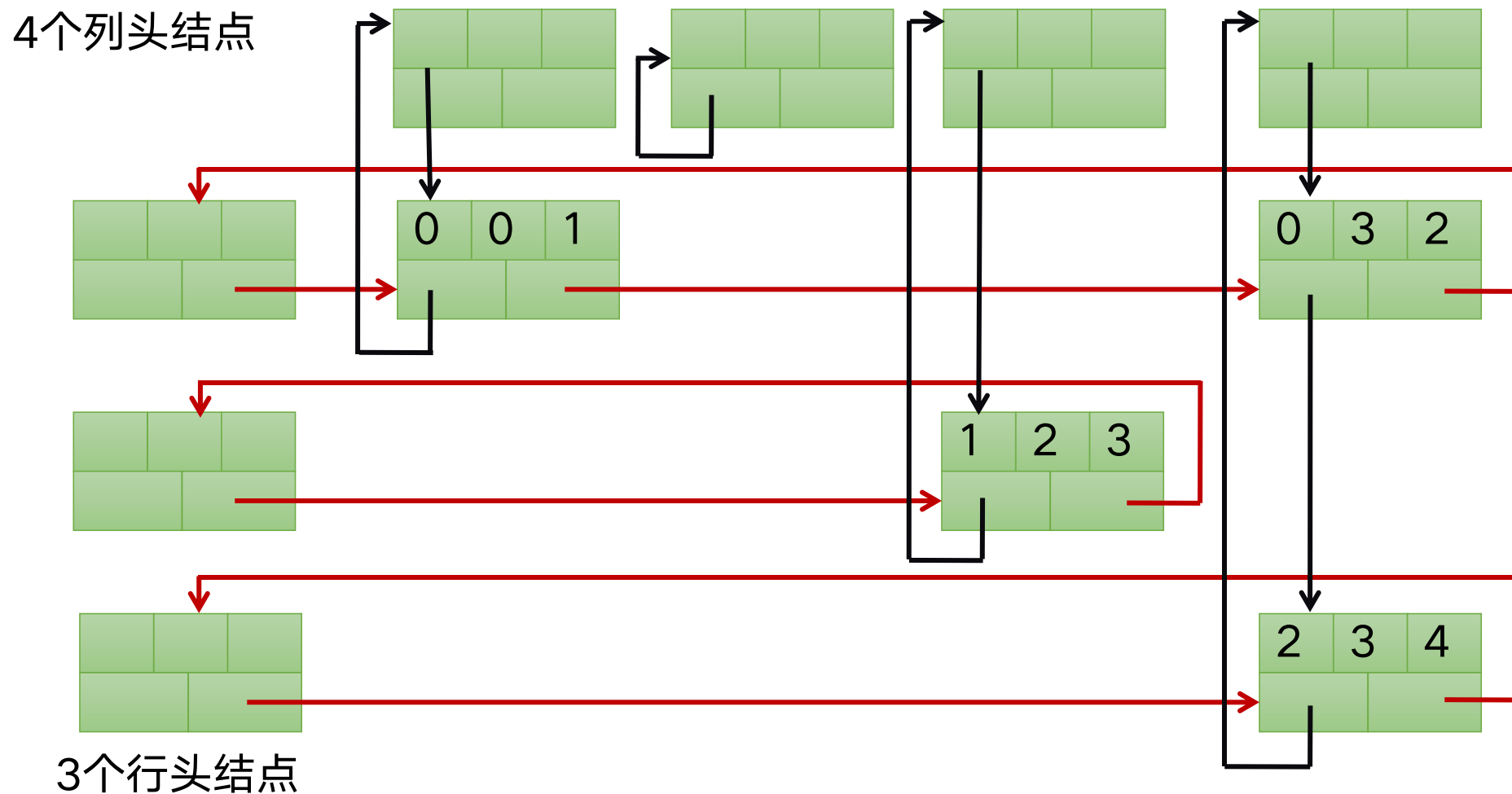


3个行头结点

6.2 稀疏矩阵

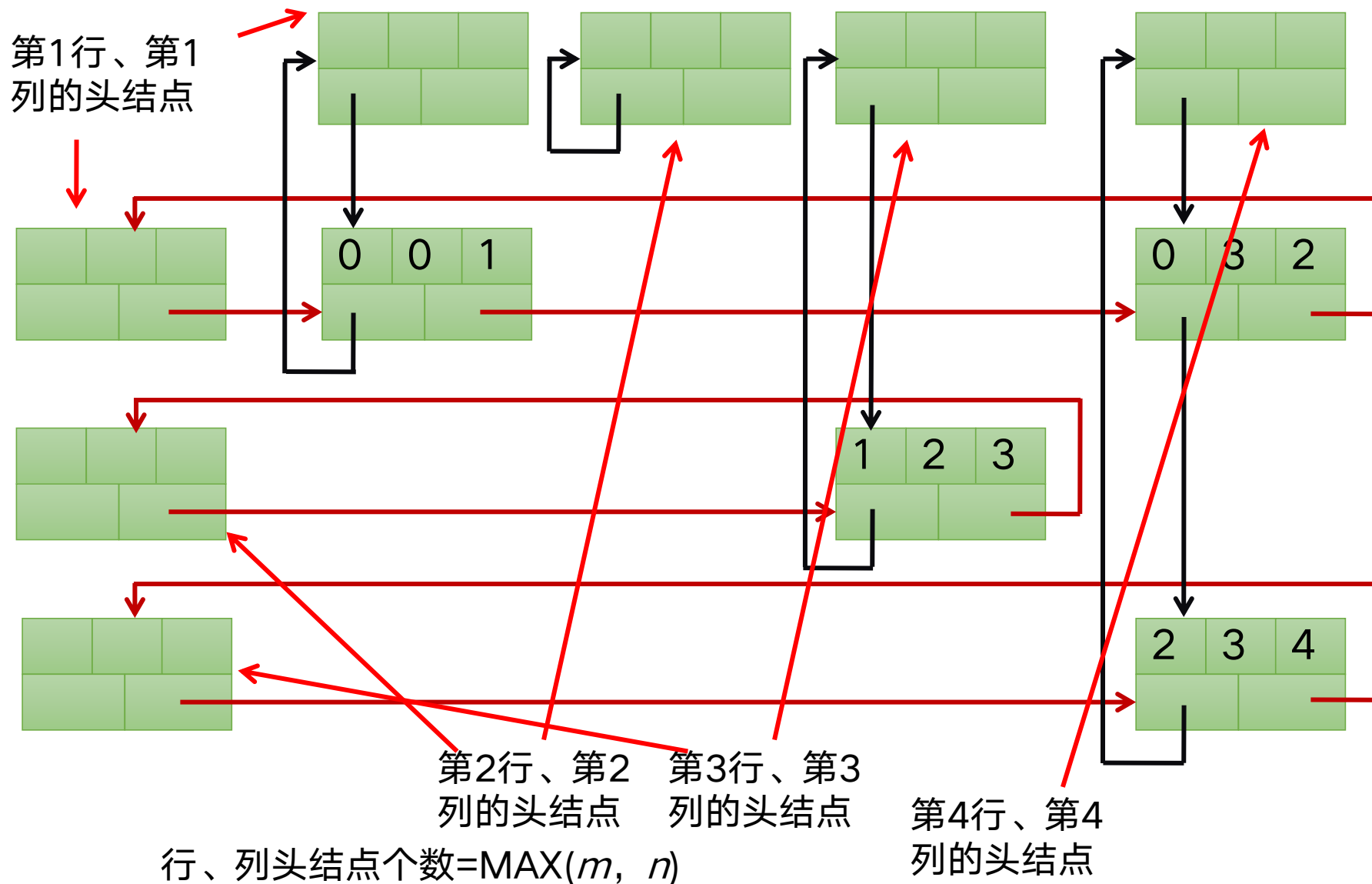
6.2.2 稀疏矩阵的十字链表表示

- **每列**的所有结点链起来构成一个带列头结点的循环单链表。以 $h[i]$ ($0 \leq i \leq m-1$) 作为第*i*列的头结点。



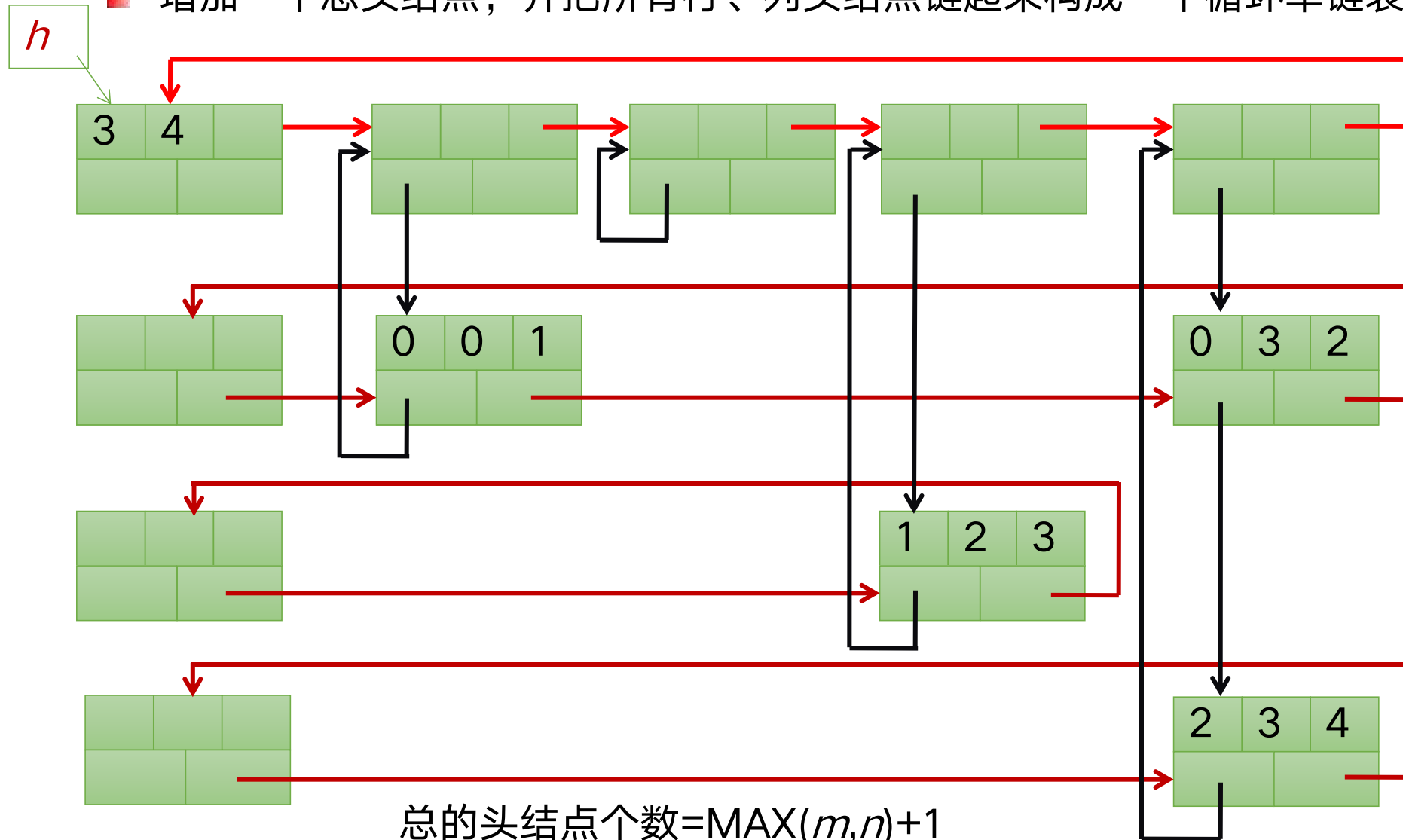
6.2 稀疏矩阵

行、列头结点共享



6.2 稀疏矩阵

■ 增加一个总头结点，并把所有行、列头结点链起来构成一个循环单链表



6.2 稀疏矩阵

6.2.2 稀疏矩阵的十字链表表示

➤ 为了统一，设计结点类型如下：



(a) 数据结点结构



(b) 头结点结构

用共用体表示

right: 指向同一行中右边的下一个非零元素
down: 指向同一列中下方的下一个非零元素

6.2 稀疏矩阵

6.2.2 稀疏矩阵的十字链表表示

示例



一个稀疏矩阵采用压缩后，和直接采用二维数组存储相比会失去（ ）特性。

A.顺序存储

B.随机存取

C.输入输出

D.以上都不对

提纲

CONTENTS

6.1 数 组

6.2 稀疏矩阵

▶ 6.3 广义表

6.3 广义表

6.3.1 广义表的定义

广义表是线性表的推广，是有限个元素的序列，其逻辑结构采用括号表示法表示如下：

$$GL = (a_1, a_2, \dots, a_i, \dots, a_n)$$

- 若 $n=0$ 时称为空表。
- a_i 为广义表的第 i 个元素。如果 a_i 属于**原子类型**，称之为广义表GL的原子。 原子类型：不可再分解的，例如整型、字符型等
- 如果 a_i 又是一个广义表，称之为广义表GL的**子表**。

$$A = (a, (b, c))$$

6.3 广义表

6.3.1 广义表的定义

广义表重要概念：

- 广义表的**长度**定义为最外层包含元素个数。
- 广义表的**深度**定义为所包括弧的重数。其中，原子的深度为0，空表的深度为1。
- 广义表GL的**表头**为第一个元素 a_1 ，其余部分 $(a_2, \dots, a_n)$ 为GL的**表尾**。一个广义表的表尾始终是一个广义表。空表无表头表尾。

6.3 广义表

6.3.1 广义表的定义

$A=()$ $\longrightarrow$ 空表，其长度为0 深度为1

$B=(e)$ $\longrightarrow$ 只含有单个原子 e 的表，其长度为1 深度为1

$C=(a, (b, c, d))$ $\longrightarrow$ 有两个元素，一个原子，一个子表， C 的长度为2
深度为2

$D=(A, B, C)=((), (e), (a, (b, c, d)))$ 深度为3

$E=((a, (a, b), ((a, b), c)))$

$\uparrow$ 有3个元素，每个元素又都是一个子表， D 的长度为3
只含有一个元素，该元素是一个子表， E 的长度为1

深度为4



6.3 广义表

6.3.1 广义表的定义

$A = () \longrightarrow$ 空表无表头表尾

$B = (e) \longrightarrow$ 表头= e , 表尾= $()$

$C = (a, (b, c, d)) \longrightarrow$ 表头= a , 表尾= $((b, c, d))$

$D = (A, B, C) = ((), (e), (a, (b, c, d)))$

$E = ((a, (a, b), ((a, b), c)))$

表头= $(a, (a, b), ((a, b), c))$, 表尾= $()$

表头= $()$, 表尾= $((e), (a, (b, c, d)))$

6.3 广义表

6.3.2 广义表的存储结构

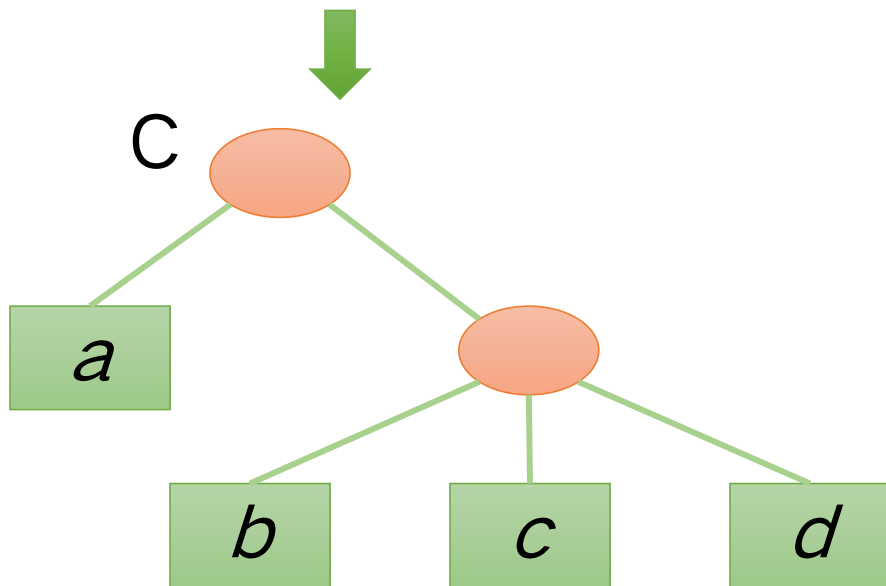
广义表的几种逻辑结构的演变，例如：

$C = (a, (b, c, d))$

括号表示

$C(a, \cdot (b, c, d))$

子表加匿名“ $\cdot$ ”

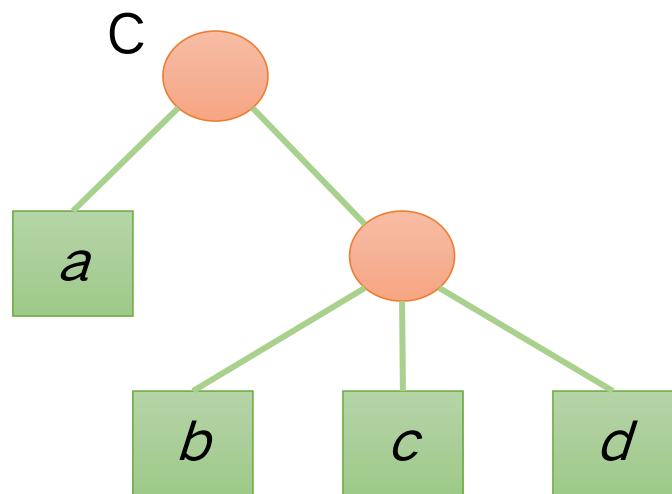


树形表示

假定一棵树的广义表表示为 **A (C, D (E, F, G) , H (I, J))** , 假若**A**为根节点, 请画出这棵树。

6.3 广义表

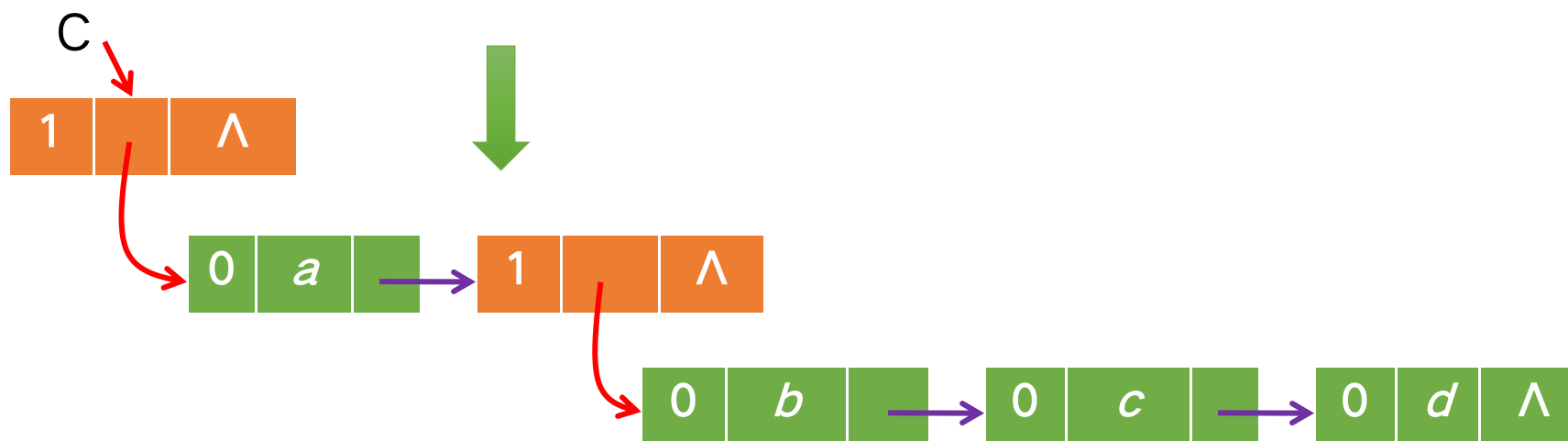
6.3.2 广义表的存储结构



结点类型:

tag	Sublist/data	link
-----	--------------	------

- 若tag=0, 表示该结点为原子结点
- 若tag=1, 表示该结点为表/子表结点



6.3 广义表

6.3.2 广义表的存储结构

广义表的结点类型GLNode

```
typedef struct lnode
{ int tag;                //结点类型标识
  union
  { ElemType data;        //存放原子值
    struct lnode *sublist; //指向子表的指针
  } val;
  struct lnode *link;      //指向下一个元素
} GLNode;                  //广义表的结点类型
```

指向兄弟

指向第一个孩子，长子